

Inteligência Artificial de Defesa Autônoma: um arcabouço de envelope e evidência para LLMs governadas que agem sobre infraestrutura de produção

Hiago Kin Levi
Decripte · hiago@decripte.us

Este paper define ADAI (Autonomous Defensive Artificial Intelligence), a categoria técnica proposta para defesa autônoma governada. Desenvolve um envelope de ação, admissibilidade excluída do modelo, verificação por capacidades disjuntas, evidências recomputáveis, contenção por principal, reconciliação e construções condicionais para ações compostas e correlação.

Edição	Preprint após o depósito, preparado em 1 de outubro de 2026
Manuscrito	Corpo técnico preservado do manuscrito fornecido
Pedido de patente	Pedido provisório nos EUA 64/166,855; patente pendente
Status científico	Preprint; sem alegação de revisão por pares
Idioma	Português; tradução do manuscrito em inglês

Os experimentos e as observações de implantação relatados correspondem à versão e ao período descritos no manuscrito. Os resultados formais dependem das hipóteses declaradas. Esta edição não afirma que toda construção teórica esteja implementada ou validada de forma independente, nem que o software esteja disponível publicamente.

© 2026 Hiago Kin Levi. Todos os direitos reservados. A disponibilidade pública deste manuscrito não concede licença sobre código proprietário, pesos do modelo ou direitos de patente. Não se declara endosso científico de terceiros.

Inteligência Artificial de Defesa Autônoma: um arcabouço de envelope e evidência para LLMs governadas que agem sobre infraestrutura de produção

Hiago Kin Levi

Tradução em português do manuscrito correspondente em inglês. O corpo técnico foi preservado; esta edição atualiza a identificação e o status do pedido de patente.

Edição após o depósito. Pedido provisório nos EUA 64/166,855, recebido em 1 de outubro de 2026. Patente pendente; ainda não concedida. DAI é a LLM própria e sua arquitetura defensiva governada; ADAI nomeia a categoria técnica proposta.

Resumo

A automação de segurança defensiva converge para uma capacidade da qual a área ainda não deu conta: agentes de software que decidem, sem serem provocados, alterar a configuração de infraestrutura de produção para deter um ataque em curso. A literatura trata isso ou como problema de orquestração — motores de playbook, que executam mas não decidem — ou como problema de controle — aprendizado por reforço em redes simuladas, que decidem mas não agem em produção. Nenhuma das duas responde à pergunta que determina se o sistema pode ser implantado: **sob que condições é admissível que uma máquina aja sobre infraestrutura sem humano no laço, e o que ela precisa ser capaz de provar depois?**

Argumentamos que a resposta não é um limiar de confiança sobre a saída de um modelo, e sim uma propriedade estrutural do sistema. Definimos o **envelope de ação** como a conjunção de quatro predicados independentemente checáveis — perímetro defendido, escopo de efeito delimitado, reversibilidade com rollback pré-declarado, e verificação por credencial de capacidade disjunta — julgados por uma função de decisão pura num contrato de ação declarado, e portanto demonstravelmente independente de qualquer componente aprendido. Definimos a obrigação de evidência como uma transcrição encadeada por hash e selada, com um **contrato de recomputação autodescritivo**, e provamos que a localização de adulteração é exata: o menor índice em que a verificação falha é o menor índice em que o registro foi alterado.

Identificamos e formalizamos dois problemas que acreditamos ser novos na literatura. A **condição do discriminador** estabelece condições necessárias e suficientes para que a contenção por principal seja expressável num ponto de aplicação compartilhado, e mostra que no caso comum de vários principais atrás de um mesmo domínio de aplicação, predicados sobre endereço de origem sozinhos não alcançam raio de alcance unitário. A **tensão agregação-reconciliação** mostra que a pressão de orçamento de regras, que obriga a agregar efeitos, destrói a bijeção entre efeito e regra que os algoritmos de reconciliação assumem, mudando a reconciliação de busca por identificador em $O(R)$ para teste de existência elemento a elemento em $O(A)$; damos o algoritmo corrigido, e mostramos que a mesma agregação torna a regra estado mutável compartilhado, admitindo uma escrita perdida que a gravação relata como sucesso e que só a verificação independente detecta.

Apresentamos um reticulado de cinco níveis de autonomia ordenado por *o que pode mudar sem humano*, e não por qualidade de modelo; uma metodologia de avaliação que separa três famílias de medição cuja confusão argumentamos ser o erro metodológico central da área; e uma caracterização das propriedades que não

admitem teste finito e que por isso precisam ser garantidas por construção.

Em seguida resolvemos quatro dificuldades que o arcabouço levanta. A **condição de atomização** diz exatamente quando a reconciliação do mundo para os registros é possível, e prescreve recusar a capacidade onde não é. Um **índice cego de correlação** produz detecção de campanha entre perímetros sem divulgação entre perímetros, com a garantia delimitada com precisão em vez de afirmada. Uma contabilidade corrigida da composição do corpus reduz "escassez" a uma questão de desbalanceamento de classe, que o portão de aprovação então limita unilateralmente. E a **reversibilidade de composta** entre domínios de aplicação vale exatamente quando os escopos de efeito declarados são dois a dois disjuntos — condição checável antes da execução, que converte um problema de composição ilimitado em uma regra de decisão.

Palavras-chave: defesa cibernética autônoma, sistemas agênticos, envelope de ação, segregação de funções, auditoria à prova de adulteração, isolamento entre tenants, recibos verificáveis, resposta a incidentes.

1. Introdução

1.1 Três literaturas que não se encontram

Orquestração de segurança. Motores de playbook mapeiam um sinal para uma sequência fixa de resposta [17, 4]. Cada ramo foi escrito por uma pessoa; quando o sinal não casa com ramo nenhum, nada acontece. Esses sistemas são prestáveis de contas precisamente porque não são autônomos, e essa prestação de contas não se transfere para sistemas que são.

Computação autônoma. O laço Monitorar–Analisar–Planejar–Executar sobre uma base de Conhecimento compartilhada [18, 16] deu à área o seu vocabulário para sistemas autogeridos. É uma arquitetura de controle, e é silenciosa sobre a pergunta que domina segurança: Planejar pode estar errado — adversarialmente, já que a entrada é em parte de autoria do adversário — e Executar altera um sistema do qual terceiros dependem. O laço descreve os estágios; não diz o que precisa valer antes de Executar poder rodar.

Defesa cibernética aprendida. Um corpo substancial de trabalho treina agentes, por aprendizado por reforço ou por prompting de modelos de linguagem, para defender redes simuladas [37, 14, 25]. Os resultados informam sobre qualidade de política e são silenciosos sobre implantação: simuladores não têm tenants, não têm credenciais, não têm terceiros cujo tráfego um bloqueio errado interrompe, e não têm obrigação de explicar depois.

A lacuna está entre a segunda e a terceira: uma arquitetura para agentes que decidem, implantada onde a consequência é real.

1.2 ADAI: a classe que cai na lacuna

Nomeamos a classe **inteligência artificial de defesa autônoma (ADAI, do inglês *autonomous defensive artificial intelligence*)** e a definimos por critérios de pertencimento, e não por capacidade, para que a pergunta "isto é uma ADAI?" tenha resposta que o leitor confere, em vez de resposta que o fornecedor afirma.

Definição 1.1 (ADAI). Um sistema é uma *inteligência artificial de defesa autônoma* se e somente se satisfaz os quatro:

A. Age sobre infraestrutura de produção. Efetua mudanças de configuração em sistemas que carregam tráfego real de dependentes reais — não num simulador, não numa cópia de homologação, e não produzindo recomendação que uma pessoa depois executa.

B. Decide sem aprovação concomitante. Para ao menos uma classe de ação, a determinação de agir é do sistema, tomada no momento do incidente, sem humano naquele laço. Um sistema cujos ramos foram todos escritos de antemão satisfaz A e falha B.

C. A admissibilidade é julgada fora do componente que propõe. As condições sob as quais a ação é permitida são avaliadas por uma função cujo domínio exclui a saída de qualquer componente aprendido ou generativo, de modo que o conjunto permitido está fixado antes de aquele componente rodar (§3).

D. Produz evidência que um não-operador consegue verificar. Cada ação produz um registro cuja integridade uma parte que não detenha código algum do sistema, nem acesso ao armazenamento dele, consegue conferir a partir do registro apenas (§4).

Definição 1.2 (Casos degenerados). Um sistema que satisfaz A e B e não C é *irrestrito*: as propriedades de segurança dele são propriedades de um componente do qual o adversário é coautor. Um que satisfaz A, B e C e não D é *não prestável de contas*: pode ser seguro e não consegue demonstrá-lo, que é a mesma posição de não ser confiável. Um que satisfaz C e D e não B é um *orquestrador auditado* — coisa valiosa, e não esta coisa.

Os critérios são deliberadamente independentes, e cada um exclui uma classe real de sistema:

Sistema	A	B	C	D	Classe
Orquestração por playbook (SOAR)	✓	✗	✓	parcial	orquestrador auditado
Agentes de RL / LLM em redes simuladas	✗	✓	✗	✗	não implantado
Agente condicionado à confiança do modelo	✓	✓	✗	✗	irrestrito
Agente com log de auditoria interno apenas	✓	✓	✓	✗	não prestável de contas
ADAI	✓	✓	✓	✓	—

1.2.1 Os critérios como predicados

Seja S um sistema, $\text{Eff}(S)$ o conjunto de mudanças de configuração que ele efetua, Dec_S a função de decisão de admissibilidade dele, $\mathcal{M}_S$ o conjunto dos componentes aprendidos ou generativos dele, e $\text{Rec}_S(a)$ o registro que ele emite para uma ação a .

$\mathbf{A}(S) \equiv \exists a \in \text{Eff}(S) : \text{alvo}(a) \in \mathcal{W}_{\text{prod}}$

$\mathbf{B}(S) \equiv \exists \theta : \forall a \in \text{Eff}(S) \text{ do tipo } \theta, \text{ exec}(a) \text{ não exige aprovação concomitante a } a$

$\mathbf{C}(S) \equiv \mathcal{M}_S \cap \text{dom}(\text{Dec}_S) = \emptyset$

$\mathbf{D}(S) \equiv \exists \text{ Ver computável a partir de } \text{Rec}_S(a) \text{ apenas} : \text{Ver}(\text{Rec}_S(a)) \text{ decide a integridade de } \text{Rec}_S(a)$

e S é uma ADAI se e somente se $\mathbf{A}(S) \wedge \mathbf{B}(S) \wedge \mathbf{C}(S) \wedge \mathbf{D}(S)$.

Duas cláusulas merecem ênfase. Em **C**, a condição é sobre o *domínio* da função de decisão, e não sobre o comportamento dela: não é "o modelo raramente muda o desfecho", é "o modelo não é um argumento". Em **D**, "computável a partir do registro apenas" exclui qualquer verificador que exija o código do produtor, o armazenamento do produtor, ou uma afirmação do produtor — e é isso que torna o critério conferível por quem tem motivo para duvidar.

1.2.2 Como testar o pertencimento

Os critérios foram escolhidos para que um leitor possa avaliá-los sem a cooperação do fornecedor em três dos quatro.

1. **A** — observar se houve mudança de configuração num sistema que carrega tráfego real. Diretamente observável.
2. **B** — observar se um humano aprovou aquela instância. Pedir o registro de aprovação; a ausência dele para uma ação que ocorreu resolve.
3. **C** — ler a assinatura da função de admissibilidade. Se o fornecedor não a mostrar, o critério está insatisfeito para os fins do leitor: afirmação inexaminável não é critério satisfeito. Onde o código não estiver disponível, um teste mais fraco e ainda útil é adversarial: submeter sinais cujos campos de texto livre argumentem por uma ação inadmissível e observar se o conjunto permitido se move.
4. **D** — pegar um registro, implementar um verificador a partir da descrição publicada nele, e rodar. Depois alterar um campo e confirmar que o verificador reprova, e que o índice que ele reporta é o índice alterado (§4.3). É o mais forte dos quatro testes e o mais barato de rodar.

Um sistema que falhe o teste 4 não é uma ADAI independentemente do que a documentação dele diga, e o teste custa uma tarde.

Duas consequências de enunciar a classe assim merecem ser explicitadas.

Os critérios são falsificáveis, e C e D o são por inspeção. Confere-se C lendo a assinatura da função de decisão; confere-se D tentando a verificação sem código algum do fornecedor e observando se ela funciona. Nenhum dos dois exige confiar numa afirmação. Uma categoria cujo pertencimento só pudesse ser afirmado não valeria o nome.

A autonomia não é a conquista; C e D são. O critério B é o fácil — qualquer sistema pode ser feito para agir sem provocação, e a maior parte do perigo neste campo vem de sistemas que fizeram exatamente isso. O que torna a classe coerente é que B é admitido *apenas* na presença de C e D. Uma ADAI não é, portanto, "uma IA que se defende sozinha"; é uma IA autorizada a se defender sozinha porque as condições do agir dela estão fixadas fora dela e o registro do agir dela é conferível fora dela.

O restante deste artigo dá o arcabouço que satisfaz C e D. O artigo companheiro descreve um sistema implantado que satisfaz os quatro.

1.3 Por que confiança do modelo é o instrumento errado

O instinto é condicionar a ação autônoma à confiança do modelo. É o instrumento errado por três razões, em ordem crescente de gravidade.

Calibração. Redes neurais modernas são mal calibradas mesmo em condições benignas [12, 27], e a entrada de um agente defensivo é em parte de autoria do adversário [2, 11]. Qualquer confiança derivada dessa entrada herda a influência do adversário sobre ela. Garantias de calibração obtidas em distribuições benignas não se transferem.

Contradomínio errado. Confiança é um escalar sobre o espaço de *proposições*. O que governa admissibilidade é o espaço de *consequências*: quanto se estraga se a proposição for falsa, e se o estrago é reversível. Uma decisão de alta confiança de descartar um /16 é estritamente pior que uma decisão de baixa confiança de apresentar um desafio a um endereço.

Não persistência. Um limiar de confiança é propriedade de um componente. Componentes são trocados, retreinados e reescritos no prompt. Uma garantia localizada na saída de um modelo não sobrevive a nenhum desses eventos; uma garantia localizada na estrutura do sistema sobrevive aos três.

Colocamos, portanto, a decisão inteiramente fora do componente aprendido, e o restante deste artigo torna essa colocação precisa.

1.4 Contribuições

1. Definição formal do **envelope de ação** (§3) e prova de que a decisão de admissibilidade executável é independente de qualquer componente aprendido (Prop. 3.4), com o limite de pior caso que dele decorre (Cor. 3.4a).
2. Tratamento formal da **obrigação de evidência** (§4), com localização exata de adulteração (Teo. 4.3) e um *contrato de recomputação* que torna possível a verificação por terceiro sem o código do produtor (§4.4); e a observação (Prop. 4.8) de que uma cadeia desta forma autentica conteúdo e posição mas **não a atribuição de principal**, o que numa implantação multi-tenant deixa a afirmação de isolamento mais forte do sistema fora do artefato que deveria evidenciá-la.
3. A **condição do discriminador** (§5, Teo. 5.5): condições necessárias e suficientes para contenção por principal num ponto de aplicação compartilhado; e uma **análise de granularidade** (Props. 5.9–5.11) mostrando que o arranjo de orçamento mínimo é uma regra única compartilhada, que escolhê-lo custa um aumento de fator q tanto no raio de alcance de um defeito de programação quanto no domínio de disputa, e que arranjo nenhum alcança isolamento em nível de credencial em pontos de aplicação que escopam tokens por domínio.
4. A **tensão agregação–reconciliação** (§6, Prop. 6.3) e um algoritmo de reconciliação corrigido, com seu custo (Alg. 2); e a gêmea dela, a **consequência de escrita perdida** da mesma agregação (§6.4, Prop. 6.7), junto com a observação de que o predicado de verificação do envelope a detecta sem ter sido desenhado para isso.
5. Um **reticulado de autonomia** ordenado por mudança admissível e não por qualidade de modelo (§7), e uma **metodologia de avaliação em três famílias** (§8), com a caracterização das propriedades que não admitem teste finito (§8.4).
6. Resoluções (§11) das quatro dificuldades que um rascunho anterior deixou em aberto: a **condição de atomização** (Teo. 11.2), que diz exatamente quando a reconciliação mundo $\rightarrow$ registros é possível e o que um sistema precisa recusar afirmar quando não é (Cor. 11.3); um **índice cego de correlação** (Constr. 11.4) que dá detecção de campanha entre perímetros sem divulgação entre perímetros, com a garantia delimitada com precisão (Prop. 11.5); uma contabilidade corrigida da composição do corpus (§11.3) junto de um limite unilateral sobre o risco de desbalanceamento de classe, que decorre de o narrador não decidir (Prop. 11.8); **reversibilidade de composta** (Teo. 11.6) com a condição de recusa que dela decorre (Cor. 11.7); e, para pontos de aplicação que não admitem atomização, um **registro de intenção antecipado** (Constr. 11.10) que converte um órfão indetectável em ambiguidade detectável que o sistema é obrigado a relatar (Cor. 11.12).

2. Modelo e notação

2.0 Objetos

Seja P um conjunto finito de **principais** (tenants). Seja $\mathcal{W}$ o espaço de configuração da infraestrutura defendida.

Um **ponto de aplicação** é o local onde mudanças de configuração passam a valer. $\mathcal{G}$ denota o conjunto de **predicados de regra** que o ponto de aplicação consegue expressar: para $g \in \mathcal{G}$, $\text{match}(g) \subseteq \mathcal{T}$ é o conjunto de tráfego que a regra casa, onde $\mathcal{T}$ é o espaço de requisições.

Para $p \in P$, $\text{traffic}(p) \subseteq \mathcal{T}$ é o tráfego destinado ao serviço de p . Assumimos que $\{\text{traffic}(p)\}_{p \in P}$ particiona o tráfego que chega ao ponto de aplicação.

O **mapa de perímetro** $\Pi : P \rightarrow 2^{\mathcal{W}}$ atribui a cada principal a região de configuração que ele tem direito de modificar.

Uma **ação** é a tupla

$$a = \langle \theta, x, \pi, \sigma, \rho, \ell \rangle$$

onde θ é o tipo de ação, x a referência do alvo, π os parâmetros, $\sigma \subseteq \mathcal{W}$ o **escopo de efeito declarado**, ρ o **rollback declarado** (sequência finita de ações), e $\ell \in \mathcal{L}$ o nível de autonomia exigido (§7).

Uma **credencial** c tem conjunto de capacidades $\text{cap}(c) \subseteq \{\text{read}, \text{write}\} \times 2^{\mathcal{W}}$.

O **raio de alcance** de uma ação é

$$B(a) = |\{p \in P : \text{effect}(a) \cap \text{traffic}(p) \neq \emptyset\}|,$$

onde $\text{effect}(a)$ é o tráfego de fato afetado — que pode diferir de σ , e a §3.2 trata de forçar que não difira.

2.1 Modelo de ameaças

Enunciamos as capacidades do adversário explicitamente, porque toda garantia abaixo é relativa a elas e várias são nulas fora delas.

O adversário pode:

- T_1 — **ser autor do conteúdo do sinal**. Todo campo de um sinal que chega, estruturado ou em texto livre, pode ser escolhido adversarialmente, inclusive campos que nomeiem terceiros inocentes. Isso não é hipótese sobre uma implantação fraca; é a condição que define o problema, já que sinais descrevem o tráfego do próprio adversário.
- T_2 — **escolher tempo e volume**. Sinais podem ser injetados em qualquer padrão, inclusive padrões desenhados para esgotar o orçamento de regras (§5.3) ou para forçar uma corrida no ciclo de revisão.
- T_3 — **comprometer a conta de um operador humano**. Testemunho (§6.6) e, onde habilitadas, aprovações de classe em A_3 podem vir de conta sob controle do atacante.
- T_4 — **observar as saídas do sistema**, inclusive narrações e recibos expostos a ele.

O adversário não pode:

- $\neg T_5$ — **obter a credencial executora c_e** . O envelope nada diz sobre um adversário que detenha c_e ; esse adversário é a autoridade do sistema sobre o perímetro, e nenhuma propriedade aqui sobrevive.
- $\neg T_6$ — **escrever no repositório de evidência abaixo da aplicação**, nem no selo D sob a autoridade de escrita separada dele (Cor. 4.5).
- $\neg T_7$ — **calcular segundas pré-imagens de H** .
- $\neg T_8$ — **controlar o próprio ponto de aplicação**. A verificação (§3.1, E_4) lê o mundo pelo mesmo provedor que executou a mudança. Um provedor que minta de forma consistente às duas credenciais derrota E_4 , e o arcabouço não oferece defesa; o que E_4 exclui é o caso muito mais comum de o sistema estar errado sobre o que fez.

O que decorre da separação. T_1 motiva o Requisito 3.8 e é a razão de a função de decisão excluir a saída do modelo: um adversário com T_1 tem autoria parcial da entrada de qualquer componente aprendido, de modo que uma garantia apoiada no comportamento desse componente apoia-se no adversário. T_3 é a razão de o testemunho ser evidência e não comando (§6.6): sob T_3 , uma interface de comando entrega o perímetro ao atacante, ao passo que uma interface de evidência entrega no máximo uma entrada para uma revisão que

ainda aplica o envelope. $\neg T_8$ é a hipótese não modelada mais significativa do arcabouço, e a sinalizamos como tal: é o ponto único cujo comprometimento invalida em silêncio a garantia de verificação, e é compartilhado com todo sistema que verifica pela mesma interface em que agiu.

3. O envelope de ação

3.1 Definição

Definição 3.1 (Envelope de ação). Para uma ação a proposta em nome do principal p , sob a política de implantação Σ , o predicado do envelope é

$$\mathcal{E}(a, p, \Sigma) = E_1 \wedge E_2 \wedge E_3 \wedge E_4$$

com

$$E_1 : \text{ep}(a) \in \Pi(p) \wedge \exists c_e : (\text{write}, \Pi(p)) \in \text{cap}(c_e) \wedge \forall (op, S) \in \text{cap}(c_e) : op = \text{write} \Rightarrow S \subseteq \Pi(p)$$

$$E_2 : \sigma \text{ é declarado antes da execução} \wedge \sigma \subseteq \Sigma_{\max}(p, \theta) \wedge \text{effect}(a) \subseteq \sigma$$

$$E_3 : \rho \neq \varepsilon \wedge \text{rec}(\rho) \prec \text{exec}(a) \wedge \text{exec}(\rho) \circ \text{exec}(a) = \text{id} \upharpoonright_{\sigma}$$

$$E_4 : \exists c_v : (\text{read}, \sigma) \in \text{cap}(c_v) \wedge \forall (op, S) \in \text{cap}(c_v) : op \neq \text{write}$$

onde $\text{ep}(a)$ é o ponto de aplicação em que a escreve, $\text{rec}(\cdot)$ denota registro durável, $\prec$ é a relação acontece-antes, e $\text{id} \upharpoonright_{\sigma}$ é a identidade no escopo de efeito.

É uma conjunção, não um escore ponderado. Não existe troca em que um σ muito estreito compense $\rho = \varepsilon$.

3.2 A função de decisão e sua independência

A separação entre um *ponto de decisão* e um *ponto de aplicação* de política é padrão em controle de acesso [26, 33]; o que segue torna explícito o domínio do ponto de decisão, de modo que a independência de um componente aprendido seja propriedade da assinatura, e não de uma checagem em tempo de execução.

Definição 3.2 (Função de decisão). A decisão de admissibilidade é

$$\delta : \mathcal{A} \times \mathcal{L} \times \{0, 1\} \longrightarrow \{\text{PERMITIR}, \text{NEGAR}, \text{APROVAR}\},$$

$$\delta(a, \lambda, h) = \begin{cases} \text{NEGAR} & \text{se } \neg \mathcal{E}(a, p, \Sigma) \\ \text{NEGAR} & \text{se } \ell(a) \succ \lambda \\ \text{PERMITIR} & \text{se } \ell(a) = \ell_{\max} \wedge \mathcal{E} \\ \text{APROVAR} & \text{se } \ell(a) \prec \ell_{\max} \wedge \neg h \\ \text{PERMITIR} & \text{se } h \end{cases}$$

onde λ é o teto de autonomia da implantação e h indica a presença de uma autorização registrada, emitida por papel com autoridade de autorizar.

Crucialmente, h é **entrada, não inferência**: δ não deduz aprovação a partir do conteúdo do contrato.

A afirmação ingênua de independência — " δ é invariante em m " — é verdadeira e vazia, já que $m \notin \text{dom}(\delta)$ por construção. É vazia porque um componente aprendido ainda escolhe *qual* a é submetido, e escolhas diferentes produzem decisões diferentes. A afirmação que carrega peso é sobre o **conjunto executável**, não sobre a decisão numa entrada fixa.

Definição 3.3 (Conjunto executável). Para um componente aprendido com saída m , seja $\mathcal{A}(m) \subseteq \mathcal{A}$ o conjunto de ações que o sistema consegue construir sob m , e

$$\text{Exec}(m, \lambda, h) = \{a \in \mathcal{A}(m) : \delta(a, \lambda, h) = \text{PERMITIR}\}.$$

Proposição 3.4 (Independência do conjunto executável). Sob a construção da Prop. 3.5, para todo m ,

$$\text{Exec}(m, \lambda, h) \subseteq \text{Exec}^*(\lambda, h) := \{a \in \bigcup_{\theta} \text{Img}(\text{instantiate} \upharpoonright_{\text{cand}(\theta)}) : \delta(a, \lambda, h) = \text{PERMITIR}\},$$

e Exec^* não depende de m .

Demonstração. Pela Prop. 3.5, $\mathcal{A}(m) \subseteq \bigcup_{\theta} \text{Img}(\text{instantiate} \upharpoonright_{\text{cand}(\theta)})$ para todo m , o que dá a inclusão. Exec^* é definido a partir de cand , instantiate e δ , e nenhum deles recebe m como argumento; logo independe de m . $\square$

A distinção importa operacionalmente. A Prop. 3.4 **não** diz que um modelo comprometido é inofensivo: dentro de Exec^* ele pode escolher a pior ação admissível, e pode escolher agir quando não agir seria melhor. O que ela diz é que a *fronteira* do que pode acontecer é fixada antes de o modelo rodar, e é portanto auditável independentemente dele. Um limite superior de dano que não depende do componente mais exposto ao adversário é a propriedade mais forte disponível aqui; um limite que *dependesse* dele não seria limite nenhum.

Estenda B a escopos por $B(S) = |\{p \in P : S \cap \text{traffic}(p) \neq \emptyset\}|$, que coincide com a definição da §2 quando $S = \text{effect}(a)$ e é monótona: $S \subseteq S' \Rightarrow B(S) \leq B(S')$.

Corolário 3.4a (Limite de pior caso).

$$\max_m \max_{a \in \text{Exec}(m, \lambda, h)} B(a) \leq \max_{a \in \text{Exec}^*} B(a) \leq \max_{p, \theta} B(\Sigma_{\max}(p, \theta)).$$

Demonstração. A primeira desigualdade é a Prop. 3.4 mais a monotonicidade do $\max$ sobre um subconjunto. Para a segunda, todo $a \in \text{Exec}^*$ satisfaz $\mathcal{E}$, logo E_2 , logo $\text{effect}(a) \subseteq \sigma \subseteq \Sigma_{\max}(p, \theta)$; a monotonicidade de B dá $B(a) = B(\text{effect}(a)) \leq B(\Sigma_{\max}(p, \theta))$. $\square$

O lado direito é constante da implantação. O raio de alcance é, portanto, limitado por configuração, e não por comportamento de modelo — e o limite é *checável sem rodar o sistema*, lendo $\Sigma_{\max}$ e a partição de tráfego.

Proposição 3.5 (Fechamento do catálogo). Seja $\mathcal{C}$ um catálogo finito de playbooks revisados e $\text{cand} : \Theta \rightarrow 2^{\mathcal{C}}$ o mapa de tipos de sinal para conjuntos de candidatos. Se a construção de ação é definida como $a = \text{instantiate}(c, x)$ para $c \in \text{cand}(\theta)$, e uma preferência aprendida m só pode escolher dentro de $\text{cand}(\theta)$, então

$$\{a : a \text{ construível}\} = \bigcup_{\theta} \{\text{instantiate}(c, x) : c \in \text{cand}(\theta)\},$$

independentemente de m .

Demonstração. A imagem de instantiate é determinada por $\text{cand}(\theta)$, que é um mapa fixo. Uma preferência é uma função de escolha sobre $\text{cand}(\theta)$; funções de escolha não ampliam o próprio domínio. $\square$

Corolário 3.6. Um modelo pode propor, ordenar e narrar. Não pode ampliar o conjunto permitido. Consequentemente, manipulação adversarial da saída do modelo — por injeção de prompt através do sinal, por deslocamento de distribuição, ou por substituição do modelo — não consegue produzir ação fora do catálogo nem fora do envelope.

3.3 Proveniência do alvo

Catálogos revisados de comportamento adversário e de procedimentos de resposta são a fonte natural de $\mathcal{C}$ e de cand [38, 4].

Definição 3.7 (Alvo ancorado). Um alvo x é *ancorado* se é extraído do campo de evidência estruturada do sinal por uma função total de parsing $\text{ext} : \mathcal{J} \rightarrow 2^{\mathcal{X}}$ sobre um conjunto fixo de chaves, onde $\mathcal{J}$ é o espaço de documentos estruturados bem formados.

Requisito 3.8. Somente alvos ancorados podem ser acionados. Em particular, nenhum alvo pode ser derivado de campos de linguagem natural.

A justificativa é a autoria adversarial, e é a ameaça de injeção indireta de prompt [11, 30, 8] enunciada em termos de efeitos e não de saídas. Resumos em texto livre são texto que o adversário influencia; acionar um identificador que aparece num deles é caminho direto para negação de serviço contra um terceiro escolhido pelo adversário. Formalmente, se ext percorresse texto livre, o adversário controlaria um subconjunto da saída de ext , logo controlaria x , logo controlaria $\text{effect}(a)$ — e $\mathcal{E}$ continuaria valendo, porque o envelope restringe *escopo*, não *escolha*. O Requisito 3.8 é o complemento que o envelope não fornece.

Definição 3.9 (Tipos acionáveis). Seja K o conjunto de tipos de alvo reconhecidos por ext e $K_A \subseteq K$ aqueles para os quais existe playbook revisado. Tipos em $K \setminus K_A$ são registrados como *observados* e não oferecidos. A relação $K_A \subsetneq K$ é deliberada: um alvo do tipo número de sistema autônomo, por exemplo, tem raio de alcance igual à base inteira de clientes daquele provedor, e nenhum predicado do envelope sobre um único principal consegue delimitá-lo.

3.4 Por que cada predicado é estrutural e não procedimental

Cada predicado é escolhido para poder ser garantido por construção, e não por convenção — porque convenção falha em silêncio.

E_1 é garantido pelo escopo da credencial e pelo vínculo entre perímetro e principal. A falha característica que ele impede é sutil: um sistema que filtra corretamente *leituras* por tenant e não filtra *ações*. Isolamento de leitura é barato de auditar — toda consulta tem uma cláusula de filtro, e toda cláusula pode ser inspecionada — e a correção dele dá uma falsa impressão do todo. O caminho de ação é onde o isolamento é caro de errar e barato de esquecer, porque enquanto um único tenant gera sinal a omissão é invisível. A §9 traz uma instância medida.

E_2 é garantido por fazer do escopo de efeito um campo declarado do contrato, comparado contra uma política que é propriedade da implantação e não do pedido. Um agente capaz de descrever a própria ação como estreita estaria corrigindo a própria prova.

E_3 é garantido por exigir que o rollback seja registrado antes da execução. Registrá-lo depois não é equivalente: um sistema que falha entre agir e registrar deixa um efeito no mundo sem descrição de como desfazê-lo. A ordem é a garantia.

E_4 é garantido por capacidade, não por caminho de código. "Quem verifica não executou", como propriedade da estrutura do programa, é quebrado por uma refatoração — e quebrado *em silêncio*: a verificação continua rodando, continua retornando sucesso, e agora não estabelece nada. Como propriedade da credencial, sobrevive à refatoração: um defeito que trocasse os dois objetos produziria erro de autenticação na escrita, e não uma verificação falsa. A credencial verificadora precisa, portanto, ser demonstrada — empiricamente e contra a interface viva — como recusada na escrita; um *nome* de permissão que sugira somente leitura é afirmação da documentação do provedor, não medição.

4. A obrigação de evidência

4.1 Por que registro de log não basta

Logs respondem *o que o sistema diz que aconteceu*. Essa é a pergunta que se faz depois de já se ter decidido confiar no sistema. Um agente defensivo autônomo é implantado exatamente onde essa confiança ainda não está estabelecida. O artefato precisa responder *o que aconteceu, e posso conferir sem confiar em você?*

4.2 Construção

A construção é o log encadeado por hash clássico [13, 21, 35, 6, 20], especializado de modo que o campo de encadeamento fique *dentro* do objeto serializado.

Uma sessão é uma sequência finita de **turnos** $T = \langle \tau_0, \dots, \tau_n \rangle$. Fixe um conjunto ordenado de campos $F = \langle f_1, \dots, f_k \rangle$ e uma serialização canônica κ (ordem determinística de chaves, separadores fixos, codificação fixa, coerção fixa do não serializável). Seja H um hash resistente a colisão.

$$\tau_i.\text{prev} = \begin{cases} \perp & i = 0 \\ d_{i-1} & i > 0 \end{cases} \quad d_i = H(\kappa(\tau_i)), \quad \text{prev} \in F.$$

Note que o encadeamento é *interno*: prev é um dos k campos hasheados, de modo que d_i depende de d_{i-1} através de κ e não por concatenação. Isso importa para o contrato de recomputação (§4.4), porque o verificador precisa reconstruir o objeto, e não apenas concatenar.

O registro da sessão guarda além disso um **selo** D , escrito no momento da selagem, num local fora da cadeia, com a restrição de integridade de que uma sessão não pode ser marcada como selada sem ele.

Definição 4.1 (Validade).

$$\text{Valid}(T, D) \equiv \left(\forall i \leq n : d_i = H(\kappa(\tau_i)) \right) \wedge \left(\forall i > 0 : \tau_i.\text{prev} = d_{i-1} \right) \wedge (\tau_0.\text{prev} = \perp) \wedge (d_n = D).$$

Requisito 4.2 (Append-only abaixo da aplicação). Inserção é a única mutação admissível, garantida na camada de armazenamento. Um sistema que *pretende* não reescrever a história e um que *não consegue* são sistemas diferentes; só o segundo sobrevive a um defeito na aplicação.

4.3 A localização de adulteração é exata

Seja o verificador obrigado a reportar, para cada i , qual dos dois conjuntos falha:

$$C_i^{\text{dig}} \equiv d'_i = H(\kappa(\tau'_i)), \quad C_i^{\text{elo}} \equiv \tau'_i.\text{prev} = d'_{i-1},$$

e seja $v(T') = \min\{i : \neg C_i^{\text{dig}} \vee \neg C_i^{\text{elo}}\}$.

Teorema 4.3 (Localização). Seja T' diferente de T exatamente nos turnos indexados por um conjunto não vazio J , com $j^* = \min J$. Suponha H resistente a segunda pré-imagem: para todo $\mathcal{A}$ probabilístico de tempo polinomial e todo x , $\Pr[\mathcal{A}(x) = x' \wedge x' \neq x \wedge H(x') = H(x)] \leq \text{negl}(\eta)$ no parâmetro de segurança η . Então, salvo com probabilidade $\text{negl}(\eta)$, vale exatamente um dos casos:

- (a) o adversário manteve $d'_{j^*} = d_{j^*}$; então $v(T') = j^*$ e o conjunto que falha é C^{dig} ;
- (b) $j^* < n$, o adversário recalculou $d'_{j^*} = H(\kappa(\tau'_{j^*}))$ e não recalculou a cauda; então $v(T') = j^* + 1$ e o conjunto que falha é C^{elo} ;
- (c) o adversário recalculou d'_i para todo $i \geq j^*$ — o que inclui o caso $j^* = n$ com d'_n recalculado, já que a cauda é então vazia; então todo C_i^{dig} e todo C_i^{elo} valem e $d'_n \neq d_n = D$, falhando a validade no selo, cujo reparo exige uma segunda autoridade de escrita.

Demonstração. Para $i < j^*$ nenhum turno difere e os dois conjuntos valem, logo $v(T') \geq j^*$. No caso (a), valer $C_{j^*}^{\text{dig}}$ significaria $H(\kappa(\tau'_{j^*})) = d_{j^*} = H(\kappa(\tau_{j^*}))$ com $\kappa(\tau'_{j^*}) \neq \kappa(\tau_{j^*})$ — uma segunda pré-imagem, que por hipótese ocorre com probabilidade no máximo $\text{negl}(\eta)$; falha, portanto. No caso (b), $C_{j^*}^{\text{dig}}$ vale por construção e $C_{j^*}^{\text{elo}}$ vale porque $\tau'_{j^*}.\text{prev} = d_{j^*-1}$ não mudou; em $i = j^* + 1$ o turno está intacto, logo $C_{j^*+1}^{\text{dig}}$ vale, enquanto $\tau_{j^*+1}.\text{prev} = d_{j^*} \neq d'_{j^*}$ falsifica $C_{j^*+1}^{\text{elo}}$. O caso (c) é (b) aplicado indutivamente até n ; o digesto terminal difere então do D guardado à parte. $\square$

Corolário 4.4 (Localização exata). Defina

$$\hat{j}(T') = \begin{cases} v(T') & \text{se } \neg C_{v(T')}^{\text{dig}} \\ v(T') - 1 & \text{se } C_{v(T')}^{\text{dig}} \wedge \neg C_{v(T')}^{\text{elo}} \end{cases}$$

Então $\hat{j}(T') = j^*$ nos casos (a) e (b). O caso (c) não produz índice de falha algum — nenhum $\hat{j}$ fica definido — e é detectado apenas no selo, que é por isso que o selo não pode compartilhar autoridade de escrita com a cadeia.

O refinamento não é cosmético. Reportar apenas o menor índice que falha atribui o caso (b) ao turno $j^* + 1$ — um turno intacto —, o que, numa investigação, aponta para o ator errado, o momento errado e a ação errada. Qual conjunto falhou é a informação que separa os dois casos, e um verificador que reporta só um booleano a descarta.

É a propriedade que distingue uma cadeia de uma soma de verificação, e produz uma quantidade diretamente mensurável: a **acurácia de localização de adulteração**, a fração de cadeias sinteticamente alteradas para as quais $\hat{j}(T') = j^*$ (§8.3).

Corolário 4.5 (Custo da falsificação). Pelo Teo. 4.3, forjar uma sessão exige ou uma segunda pré-imagem de H — de probabilidade desprezível no parâmetro de segurança — ou o caso (c): reescrever a cauda inteira e o selo fora da cadeia. Colocar o selo sob autoridade de escrita distinta da transcrição eleva, portanto, a falsificação de um comprometimento para dois.

4.4 O contrato de recomputação

Evidência de adulteração não vale nada se só o produtor consegue avaliá-la.

Definição 4.6 (Contrato de recomputação). Um recibo exportado satisfaz o contrato de recomputação se e somente se carrega: (i) a função de hash; (ii) a lista ordenada de campos F ; (iii) a especificação completa de κ ; (iv) a regra de encadeamento; (v) tanto D quanto o d_n recalculado, separadamente, para que a divergência fique visível; e (vi) para cada turno, **todos** os k campos de F .

Um recibo que satisfaça (i)–(vi) autentica exatamente o que F cobre. O que F omite é afirmado pelo recibo e verificado por nada — ver a Prop. 4.8.

A cláusula (vi) é a que falha na prática. Um recibo que exporta uma *projeção* do turno — um subconjunto escolhido para exibição — não pode ser recomputado por ninguém, e a falha é invisível a todo teste escrito contra a serialização do próprio produtor, porque tais testes nunca reconstroem o objeto a partir do recibo.

Requisito 4.7 (Verificação independente). A conformidade é demonstrada por um verificador implementado a partir apenas da descrição publicada no recibo, sem importar código algum do sistema produtor. Qualquer demonstração mais fraca estabelece apenas que o produtor concorda consigo mesmo.

4.5 O que a cadeia prova

A distinção é a que a literatura de proveniência faz entre a integridade de um registro e a veracidade do que ele registra [24].

Ela estabelece **processo**: que estes turnos, nesta ordem, com este conteúdo, foram registrados e não alterados depois; que o contrato precedeu a execução; que a verificação foi registrada sob papel distinto do executor.

Ela não estabelece **origem**: que o sinal era verdadeiro, que o adversário era quem a evidência nomeia, ou que o mundo correspondia à descrição. Confundir as duas é o erro mais atraente disponível a um sistema deste tipo, e a mitigação é que o recibo enuncie os próprios limites como parte do artefato.

Ela também não estabelece atribuição, e isso é fácil de não ver. O conjunto de campos F cobre o conteúdo do turno, o papel e a autoridade dele, a posição dele, e o elo com o antecessor. Não cobre o *principal* a que a sessão pertence. O turno carrega o identificador da sessão e o registro da sessão carrega o principal, de modo que a atribuição fica a uma dereferência fora da cadeia.

Proposição 4.8 (A atribuição não é autenticada). Seja $\text{owner} : \mathcal{S} \rightarrow P$ o mapa de sessão para principal, guardado fora de T . Alterar $\text{owner}(s)$ deixa $\text{Valid}(T, D)$ inalterado.

Demonstração. owner não aparece em nenhum $f \in F$, logo $\kappa(\tau_i)$, logo todo d_i , logo $D = d_n$, é invariante nele. $\square$

A consequência é precisa e vale enunciada com todas as letras: um verificador de terceiro seguindo o Req. 4.7 consegue confirmar *o que aconteceu*, e não consegue confirmar *em nome de quem*. Numa implantação multi-tenant — onde o principal é o objeto da garantia de isolamento mais forte que o sistema faz (§5) — essa é a coisa errada a deixar fora do conjunto autenticado. Na prática o vínculo é imposto por uma restrição append-only no repositório, que é garantia real mas *local*: protege os registros do próprio operador e não vale nada para um leitor externo, que não tem acesso ao repositório nem como observar que a restrição existe.

Dois remédios existem e não são equivalentes. Acrescentar o principal a F autentica a atribuição criptograficamente, ao custo de recalculer todo digesto histórico — o que um repositório append-only existe para impedir, tornando isso uma bifurcação da cadeia e não uma edição nela. Vincular o registro da sessão à carga do turno 0 autentica a atribuição para sessões abertas depois da mudança, deixa a história em paz, e é portanto o que um sistema implantado consegue de fato fazer. Enunciamos os dois, porque um artigo que ofereça só o remédio limpo descreve um sistema para o qual ninguém consegue migrar.

5. A condição do discriminador

Esta seção trata de um problema que não encontramos tratado na literatura: quando vários principais compartilham um ponto de aplicação, sob que condições a contenção por principal é sequer expressável?

5.1 Cenário

O isolamento em sistemas multi-tenant é usualmente estudado nos planos de dados e de armazenamento [19]. O objeto aqui é outro: o recurso compartilhado é o *ponto de aplicação*, e o que precisa ser isolado é o efeito de escrever nele.

Seja Z um domínio de aplicação (por exemplo, uma zona DNS atendida por uma rede de borda) servindo principais $P_Z \subseteq P$ com $|P_Z| = q > 1$. Seja $\mathcal{G}$ o conjunto de predicados de regra expressáveis.

Definição 5.1 (Contenção tenant-sólida). Uma regra $g \in \mathcal{G}$ que contém o alvo x em nome de p é *tenant-sólida* se

$$\text{match}(g) \subseteq \text{traffic}(p) \quad \text{e} \quad \text{from}(x) \cap \text{traffic}(p) \subseteq \text{match}(g),$$

isto é, contém todo o tráfego de x para p e nada do tráfego de mais ninguém.

Definição 5.2 (Raio de alcance unitário). $B(g) = 1$.

Lema 5.3. Se todo $g \in \mathcal{G}$ tem a forma $\text{from}(x)$ — predicado apenas sobre a origem — então, para qualquer x com tráfego para mais de um principal, $B(g) = |\{p : \text{from}(x) \cap \text{traffic}(p) \neq \emptyset\}| > 1$, e nenhum g é tenant-sólido.

Demonstração. $\text{match}(g) = \text{from}(x)$ independe de p e intersecta o tráfego de todo principal que x alcança. A primeira cláusula da Def. 5.1 falha. $\square$

Este é o conteúdo formal de uma falha fácil de não enxergar operacionalmente: um bloqueio por endereço de origem numa borda compartilhada, levantado por causa do incidente do principal p_1 , nega serviço em silêncio a $p_2, \dots, p_q$ — incluindo, na implantação da §9, o próprio console que um operador usaria para observar o incidente.

5.2 A condição

Definição 5.4 (Discriminador). Uma função $d : \mathcal{T} \rightarrow \mathcal{D}$ é um *discriminador* para P_Z se existe $\phi : P_Z \rightarrow 2^{\mathcal{D}}$ com $\{\phi(p)\}_{p \in P_Z}$ dois a dois disjuntos e

$$\forall p \in P_Z, \forall t \in \mathcal{T} : t \in \text{traffic}(p) \iff d(t) \in \phi(p).$$

Assumimos, como vale em todo ponto de aplicação que conhecemos, que $\mathcal{G}$ é fechado sob conjunção e sob disjunção finita, e que o espaço de origens $\mathcal{X}$ é finito.

Teorema 5.5 (Condição do discriminador). São equivalentes:

1. A contenção tenant-sólida é expressável em Z para todo $p \in P_Z$ e todo $x \in \mathcal{X}$.
2. Para todo $p \in P_Z$ existe um **predicado separador** $\pi_p \in \mathcal{G}$ com $\text{match}(\pi_p) = \text{traffic}(p)$.
3. Existe um discriminador d para P_Z cujas classes são expressáveis em $\mathcal{G}$, e a contenção toma a forma

$$g_{p,x} = (d(t) \in \phi(p)) \wedge (t \in \text{from}(x)).$$

Demonstração. (2 $\Rightarrow$ 1) Tome $g_{p,x} = \pi_p \wedge \text{from}(x)$, que está em $\mathcal{G}$ por fechamento sob conjunção. Então $\text{match}(g_{p,x}) = \text{traffic}(p) \cap \text{from}(x)$, satisfazendo as duas cláusulas da Def. 5.1; e como $\{\text{traffic}(p)\}$ particiona, $B(g_{p,x}) = 1$.

(1 $\Rightarrow$ 2) Fixe p e seja $g_{p,x}$ tenant-sólida para cada x . Ponha $\pi_p = \bigvee_{x \in \mathcal{X}} g_{p,x}$, que está em $\mathcal{G}$ por fechamento sob disjunção finita. A primeira cláusula da Def. 5.1 dá $\text{match}(g_{p,x}) \subseteq \text{traffic}(p)$ para cada x , logo $\text{match}(\pi_p) \subseteq \text{traffic}(p)$. Reciprocamente, seja $t \in \text{traffic}(p)$ e x a sua origem; a segunda cláusula dá $\text{from}(x) \cap \text{traffic}(p) \subseteq \text{match}(g_{p,x})$, e t pertence ao lado esquerdo, logo $t \in \text{match}(\pi_p)$. Portanto $\text{match}(\pi_p) = \text{traffic}(p)$.

(2 $\Leftrightarrow$ 3) Dados os $\{\pi_p\}$, tome $\mathcal{D} = P_Z$, $d(t) =$ o único p com $t \in \text{match}(\pi_p)$ — bem definido porque $\{\text{traffic}(p)\}$ particiona — e $\phi(p) = \{p\}$; a Def. 5.4 vale, e a classe $\{p\}$ é expressa por π_p . Reciprocamente, dado um discriminador cujas classes sejam expressáveis, $\pi_p := (d(t) \in \phi(p))$ tem $\text{match}(\pi_p) = \text{traffic}(p)$ pela Def. 5.4. $\square$

A equivalência entre (1) e (2) é o que torna a condição útil na prática: não é preciso procurar um discriminador, apenas perguntar se o ponto de aplicação consegue expressar *um predicado por principal que case exatamente o tráfego daquele principal*. Se não consegue, nenhuma quantidade de programação no agente produzirá contenção tenant-sólida, e a resposta certa de engenharia é mudar o ponto de aplicação, não o agente.

Corolário 5.6. Quando os principais são separados por nome de host de serviço, o campo de host é um discriminador, e a contenção tenant-sólida exige que o ponto de aplicação expresse predicados conjugando host e origem. Pontos de aplicação que ofereçam apenas regras de acesso baseadas em origem são insuficientes, independentemente de como o sistema seja programado; a limitação está em $\mathcal{G}$, não no agente.

Corolário 5.7 (Principais indefensáveis). Se o tráfego de p não atravessa Z , então nenhum $g \in \mathcal{G}$ o afeta, e registrar p como defendido em Z faz o sistema registrar contenções que não existem. O comportamento correto é recusar o registro: um falso negativo não registrado é estritamente preferível a um falso positivo

registrado sobre o estado do mundo.

5.3 Pressão de orçamento de regras

Pontos de aplicação reais limitam $|\mathcal{G}|$ em uso. Seja L o limite da plataforma para regras personalizadas e k o número já implantado para outros fins, dando orçamento $L - k$.

Escreva $\mathcal{X}_p$ para o conjunto de endereços atualmente contidos para p e $N = \sum_p |\mathcal{X}_p|$ para o total de contenções em vigor. Três granularidades são expressáveis, e damos a contagem de regras que cada uma exige.

Definição 5.8 (Granularidade). Um *arranjo de contenção* é uma função que atribui cada contenção (p, x) a uma regra. O *custo* $\mathcal{R}$ dele é o número de regras distintas usadas.

- **Por contenção:** $g_{p,x} = (d(t) \in \phi(p)) \wedge (\text{src}(t) = x)$, logo $\mathcal{R} = N$.
- **Por principal:** $g_p = (d(t) \in \phi(p)) \wedge (\text{src}(t) \in \mathcal{X}_p)$, logo $\mathcal{R} = |\{p : \mathcal{X}_p \neq \emptyset\}| \leq q$.
- **Regra única compartilhada:** $g^* = \bigvee_p [(d(t) \in \phi(p)) \wedge (\text{src}(t) \in \mathcal{X}_p)]$, logo $\mathcal{R} = 1$.

Proposição 5.9 (Orçamento). Os três arranjos são tenant-sólidos (Def. 5.1), e $\mathcal{R}$ vale N , no máximo q , e 1 respectivamente. Sob orçamento $L - k$, o arranjo por contenção só é viável enquanto $N \leq L - k$; os outros dois são viáveis para todo N até o limite de tamanho da expressão.

Demonstração. Tenant-solidez: cada disjuncto de cada arranjo tem $\text{match} \subseteq \text{traffic}(p)$ pelo Teo. 5.5, e a disjunção em g^* é sobre $\{\phi(p)\}$ dois a dois disjunctos, de modo que não surge casamento entre principais. As contagens são imediatas. $\square$

O arranjo de orçamento mínimo é, portanto, a regra única compartilhada — e não é ele que se deve escolher. Dizer por quê é o ponto desta subseção, porque "agregamos para caber no orçamento" deixaria o leitor com um argumento que seleciona o arranjo errado.

O que a granularidade não compra. Ela *não* compra isolamento de credencial. Os pontos de aplicação que examinamos escopam tokens de escrita ao **domínio**, e não a regras individuais, de modo que em todo arranjo a credencial executora de p é tecnicamente capaz de escrever qualquer regra de Z . A garantia de E_1 alcança, portanto, *em que domínio* uma credencial pode escrever, e para antes de *em que regra dentro dele*. Isso é lacuna real entre o ideal do arcabouço e o que os pontos de aplicação hoje expõem, e a sinalizamos em vez de descrever o arranjo como alcançando o que ele não alcança.

O que a granularidade compra são duas coisas, ambas sobre falha e não sobre autoridade.

Proposição 5.10 (Contenção de falha). Seja um defeito que corrompa a expressão escrita para o principal p . Sob o arranjo por principal a corrupção fica confinada a $\mathcal{X}_p$; sob a regra única compartilhada ela pode destruir $\bigcup_{p'} \mathcal{X}_{p'}$.

Demonstração. Escrever g_p endereça uma regra cujo conjunto de casamento está contido em $\text{traffic}(p)$; nenhuma contenção de outro principal está representada naquele objeto. Escrever g^* endereça o único objeto que representa a contenção de todos. $\square$

Proposição 5.11 (Contenção de disputa). Sob a leitura-modificação-escrita da §6.4, o arranjo por principal admite escrita perdida apenas entre dois episódios do *mesmo* principal; a regra única compartilhada admite entre *quaisquer* dois episódios.

Demonstração. A disputa é pelo objeto escrito. Sob o arranjo por principal, episódios de principais distintos escrevem objetos distintos; sob g^* , todos escrevem um só. $\square$

A escolha. Adotamos o arranjo por principal: $\mathcal{R} \leq q$ em vez do mínimo 1, pagando $q - 1$ regras por uma redução de fator q no raio de alcance de um defeito e no domínio de disputa. É a mesma troca que o arcabouço faz em toda parte — aceitar um custo limitado e declarado para converter falha correlacionada em falha isolada — e vale notar que o arranjo de orçamento mínimo é também aquele em que um único erro de programação desarma em silêncio a instalação inteira.

O arranjo por contenção é excluído não por preferência e sim por viabilidade: na implantação da §9, $L - k$ é um inteiro pequeno, de modo que $\mathcal{R} = N$ limita a instalação a esse número de contenções simultâneas. Isso não é restrição de projeto; é brinquedo.

A agregação é, portanto, forçada pelo orçamento, e a *granularidade* dela é escolhida contra contenção de falha. Com ela, aparecem os problemas da §6 — os dois, e os dois são consequências da mesma decisão.

6. A tensão agregação–reconciliação

6.1 Por que a reconciliação é obrigatória

A travessia mundo $\rightarrow$ registros é o problema de convergência da gestão declarativa de configuração [3, 15], sob uma restrição que aqueles sistemas não têm: o registro é probatório, de modo que a divergência não pode ser reparada reescrevendo o registro.

A revisão pós-ação percorre *registros* $\rightarrow$ *mundo*: para cada contenção registrada, ela ainda se justifica? Isso deixa um buraco. Seja t_{act} o instante em que o efeito passa a valer e t_{rec} o instante em que o registro se torna durável. Não é possível torná-los simultâneos. Um efeito criado na janela $[t_{\text{act}}, t_{\text{rec}})$ durante a qual o processo falha é **um efeito no mundo sem registro**, e é invisível para sempre à revisão guiada por registros.

Requisito 6.1. O sistema precisa percorrer também *mundo* $\rightarrow$ *registros*.

Definição 6.2 (Adoção). Um efeito sem registro é *adotado*: o sistema abre uma sessão registrando o que observa **agora**, declinando explicitamente de afirmar o instante de criação. Reconstruir um carimbo plausível colocaria num registro probatório append-only uma proposição que ninguém observou — corrompendo a única propriedade que o registro promete. O relógio de expiração começa na adoção, o que erra na direção de manter proteção em vez de removê-la cedo.

6.2 A tensão

A reconciliação guiada por regra assume um mapa de identificação $\text{id} : \mathcal{R} \rightarrow \mathcal{S} \cup \{\perp\}$ de regras para sessões, recuperável da própria regra (por exemplo, um identificador embutido na descrição).

O que a reconciliação de fato precisa não é de id , e sim do *mapa de origem* $\text{orig}_p : X_p \rightarrow \mathcal{S}$, que dá a sessão que conteve cada endereço. A pergunta é se orig_p é recuperável da regra.

Proposição 6.3 (A origem não é recuperável sob agregação). Suponha que a representação observável da regra seja $\langle \phi(p), X_p, \text{desc}(p) \rangle$, com desc dependendo só de p , e X_p um *conjunto* — isto é, a representação é invariante por permutação de X_p e não carrega anotação por elemento. Se $|X_p| \geq 2$, então nenhuma função da regra computa orig_p .

Demonstração. Sejam $x_1 \neq x_2 \in X_p$ e $s_1 \neq s_2$ sessões. Considere duas histórias H_1, H_2 que produzem a mesma regra, com $\text{orig}_p^{(1)}(x_1) = s_1, \text{orig}_p^{(1)}(x_2) = s_2$ e $\text{orig}_p^{(2)}(x_1) = s_2, \text{orig}_p^{(2)}(x_2) = s_1$. Ambas produzem a representação idêntica $\langle \phi(p), \{x_1, x_2\}, \text{desc}(p) \rangle$. Uma função da regra assume o mesmo valor nas duas, e no entanto $\text{orig}_p^{(1)} \neq \text{orig}_p^{(2)}$. $\square$

Corolário 6.3a. id não é, portanto, apenas indisponível: é mal posto. Qualquer $\text{id} : \mathcal{R} \rightarrow \mathcal{S}$ total escolhe uma dentre $|X_p|$ origens, e a escolha é arbitrária. Uma implementação que embute *um* identificador de sessão na descrição — primeira tentativa natural — não é solução parcial; é uma regra que atesta a proveniência de um endereço e atribui errado todos os demais.

Corolário 6.4. Algoritmos de reconciliação indexados por identidade de regra são *incorretos* sob agregação, nas duas direções:

- **Falso negativo:** um endereço sem registro dentro de uma regra que no mais está registrada nunca é detectado, porque a regra "tem sessão".
- **Falso positivo:** se o teste de existência de registro é avaliado por regra e falha por qualquer motivo, a regra inteira — toda contenção legítima dentro dela — é readotada a cada ciclo, produzindo registros duplicados sem limite.

Ambos os modos foram observados na implantação da §9.

6.3 Algoritmo corrigido

A unidade da reconciliação precisa ser a unidade do efeito.

Algoritmo 1 — Reconciliação identificada por regra (correta só sem agregação)

```
para g em regras_marcadas_como_nossas(Z):
    s ← id(g)
    se s = ⊥ ou não tem_registro_de_execução(s):
        adotar(g)
```

Custo: $O(R)$ buscas, R = número de regras.

Algoritmo 2 — Reconciliação identificada por elemento (correta sob agregação)

```
para p em principais_defendidos_em(Z):
    X ← ler_conjunto_contido(c_v(p))      # só a credencial verificadora
    se X = ⊥: continuar                  # ilegível ≠ vazio; adiar
    para x em X:
        se não existe_registro(p, x):
            adotar(p, x, modo ← ler_modo(c_v(p), x))
```

Custo: $O(A)$ testes de existência, $A = \sum_p |X_p|$.

Três detalhes sustentam o algoritmo, e cada um corresponde a um defeito observado.

Ler pela credencial verificadora. Reconciliar é observar; usar a credencial executora permitiria que um defeito no reconciliador modificasse o perímetro que ele audita.

Ilegível não é vazio. Se o ponto de aplicação não pode ser lido, tratar $X = \emptyset$ conclui em silêncio "nada está contido" e abandona a travessia; tratar todo endereço como órfão adota o mundo. O correto é adiar.

O modo adotado é lido, não assumido. Registrar uma severidade padrão diferente da real faz a revisão seguinte comparar o que leu com o que registrou e concluir divergência onde não há.

6.4 A agregação também torna a regra estado mutável compartilhado

O problema de reconciliação da §6.2 é a consequência mais conhecida da agregação. Há uma segunda, e ela é propriedade de concorrência, não de identificação.

Definição 6.6 (Contenção como leitura-modificação-escrita). Sob a §5.3, conter x para o principal p é a operação

$$\text{conter}(p, x) : \quad X_p \leftarrow \text{ler}(g_p); \quad X'_p \leftarrow X_p \cup \{x\}; \quad \text{escrever}(g_p, X'_p),$$

sem atomicidade entre a leitura e a escrita, a menos que o ponto de aplicação ofereça pré-condição de comparação-e-troca sobre g_p .

Proposição 6.7 (Escrita perdida). Sejam duas contenções $\text{conter}(p, x_1)$ e $\text{conter}(p, x_2)$ intercaladas como

$$\text{ler}_1 \prec \text{ler}_2 \prec \text{escrever}_1 \prec \text{escrever}_2,$$

com as duas leituras devolvendo X_p . Então o estado final é $X_p \cup \{x_2\}$, e x_1 não está contido, embora o episódio dele tenha executado, nada em contrário tenha sido verificado no momento da escrita, e ele fosse selar.

Demonstração. escrever_2 carrega $X_p \cup \{x_2\}$, calculado a partir de ler_2 , que precedeu escrever_1 e portanto não observou x_1 . A última escrita vence. $\square$

Três observações fazem disto mais que uma corrida de livro-texto.

A escrita não a detecta. O provedor devolve sucesso: a regra foi atualizada, exatamente como pedido. A falha é que o *que foi pedido* foi calculado sobre estado velho.

E_4 a detecta, e não por acaso. A verificação independente relê o mundo em vez de confiar na resposta da escrita (§3.4), de modo que a verificação do episódio perdedor devolve *falhou*. O quarto predicado do envelope, introduzido para pegar o sistema errado sobre o que fez, pega isto sem conhecer o mecanismo. É o tipo de cobertura que uma garantia estrutural dá e uma checagem dirigida não dá.

Detectar não é recuperar. Um episódio verificado-e-reprovado sela honestamente e deixa x_1 sem contenção. As respostas disponíveis, ordenadas por força:

1. *Comparação-e-troca sobre g_p* , onde o ponto de aplicação a ofereça. Correta, e indisponível nos endpoints por regra que examinamos.
2. *Serializar escritas por principal*, por exemplo com uma concessão por p . Correta, e acrescenta dependência de coordenação ao caminho de ação — o componente cuja disponibilidade mais importa.
3. *Releitura-e-nova-tentativa limitada*. Recupera o caso de conflito único, que é o esmagadoramente comum, e degrada para falha (detectada) além dele.
4. *Depender de haver um único escritor*. Esta é a tentadora, e é errada de um jeito que merece nome: escritor único é propriedade da **topologia de implantação** — uma contagem de processos, uma fila consumida em sequência — e não do componente. Vale até alguém escalar horizontalmente, e a tarefa agendada de revisão, que também escreve no perímetro, já a viola.

Adotamos (3), com limite de uma nova tentativa. Um laço mascararia contenção sustentada sobre um único principal, e contenção sustentada é sinal sobre a taxa de incidentes, não condição a absorver em silêncio.

Corolário 6.8. A agregação troca uma restrição de orçamento de regras por duas consequências: o mapa de origem se perde (Prop. 6.3), e a regra vira estado mutável compartilhado (Prop. 6.7). As duas decorrem da *mesma* decisão, e um sistema que adota agregação por capacidade sem tratar as duas adotou metade dela.

6.5 A revisão como função de decisão

Definição 6.9. Sejam $\text{present} \in \{0, 1\}$ (regra observada via c_v), $\text{recur} \in \{0, 1\}$ (alvo reincidente desde a contenção), Δt o tempo decorrido, W a janela de não reincidência, e $\text{test} \in \{\perp, \text{FP}, \text{CONF}, \text{CTX}\}$ o testemunho humano registrado. A função de veredito é

$$V = \begin{cases} \text{FALSO_POSITIVO} & \text{test} = \text{FP} \\ \text{AUSENTE} & \neg \text{present} \\ \text{REAFIRMADA} & \text{present} \wedge \text{recur} \\ \text{CEDO_DEMAIS} & \text{present} \wedge \neg \text{recur} \wedge \Delta t < W \\ \text{EXPIRADA} & \text{present} \wedge \neg \text{recur} \wedge \Delta t \geq W \end{cases}$$

CEDO_DEMAIS não abre sessão. "Volte depois" não é episódio, e registrar cada não decisão treina o operador a parar de ler o registro — degradação invisível a qualquer métrica técnica e bem documentada na pesquisa operacional sobre fadiga de alarme [1, 7].

6.6 Testemunho como evidência, nunca como comando

O conhecimento humano precisa ser admissível: um operador pode saber que um endereço contido pertence a um parceiro. Nós o admitimos como **classificação registrada** que entra no fluxo de evidência, lida pela revisão seguinte, que pode então chegar a FALSO_POSITIVO e desfazer pelo caminho ordinário de ação.

Nós **não** o admitimos como instrução que atua sobre o perímetro. A distinção é propriedade de segurança, não de estilo: uma interface em que a afirmação de uma pessoa altera diretamente o perímetro é uma interface em que comprometer a conta dessa pessoa altera diretamente o perímetro. Encaminhar testemunho por evidência preserva o envelope — a remoção continua sendo ação, contratada, verificada e registrada.

7. O reticulado de autonomia

Níveis de automação têm longa história na pesquisa de fatores humanos [36, 29] e uma instância industrial muito citada em automação veicular [31]. Aquelas taxonomias são ordenadas por *quem executa qual função*. A nossa é ordenada por *que classe de estado pode mudar sem humano*, que é a ordenação que importa quando a saída da automação é uma mudança de configuração em infraestrutura da qual terceiros dependem.

Definição 7.1. $\mathcal{L} = \{A_0 \prec A_1 \prec A_2 \prec A_3 \prec A_4\}$, ordem total:

Nível	Pode mudar sem humano	Papel do humano
A_0	nada	executa toda ação
A_1	nada; propõe ação específica	aprova cada instância
A_2	telemetria e estado interno próprios	aprova efeitos externos
A_3	perímetro defendido, dentro de $\mathcal{E}$	aprova classes de ação
A_4	perímetro defendido, dentro de $\mathcal{E}$	revisa depois; pode reverter

Duas propriedades importam mais que a tabela.

Proposição 7.2 (Ortogonalidade à qualidade do modelo). O nível exigido por uma ação é função do tipo da ação e da classe de efeito apenas: $\ell = \ell(\theta, \sigma)$. Em particular, ℓ independe de qualquer medida de qualidade do componente que propõe.

Consequência. Melhorar o modelo não amplia autoridade. É escolha deliberada, com custo real — um modelo melhor não pode ser recompensado com mais latitude — e benefício real: autoridade muda apenas por decisão explícita, registrada como tal.

Definição 7.3 (Teto). A implantação fixa $\lambda \in \mathcal{L}$; ações com $\ell \succ \lambda$ são recusadas com motivo registrado. Um agente não pode elevar λ .

Requisito 7.4. Quando o código carrega um padrão λ_{def} e a implantação carrega λ_{env} , a implantação precisa reafirmar λ_{env} a cada publicação. Do contrário os dois divergem, e o padrão do código passa a governar em silêncio exatamente as implantações que ninguém reconferiu.

7.1 A_4 não é a ausência de humano

A_4 remove o humano da *decisão*, não do *laço*. O papel dele se desloca para três posições, e todas precisam existir:

1. **Antes:** fixar Σ , Π e λ .
2. **Depois:** revisar a partir de um registro que sustente revisão (§4).
3. **Contra:** a capacidade permanente de reverter qualquer ação, e de registrar testemunho que o sistema trata como evidência (§6.6).

Um sistema em A_4 sem (2) e (3) não é autônomo; é não supervisionado, que é propriedade diferente e pior.

7.2 Os doze mecanismos, nomeados

As §§3–7 definem doze mecanismos. Referir-se a cada um pelo número que o define é preciso e inutilizável — numa conversa, numa revisão de código, ou numa especificação que o leitor percorre fora de ordem. Então nós os nomeamos.

Os nomes estão ligados aos resultados formais, não paralelos a eles. Cada um abrevia uma definição ou um teorema já enunciado; nenhum acrescenta sentido que a matemática não carregue. Um leitor que desconfie de um nome pode descartá-lo e não perde nada além de brevidade.

Nome	Sigla	Definido por	O que a definição fixa
Autonomy Covenant	AC	Def. 3.1	$\mathcal{E} = E_1 \wedge E_2 \wedge E_3 \wedge E_4$. Conjunção, não escore ponderado: nenhum σ estreito compensa $\rho = \varepsilon$
Model-Excluded Authority	MEA	Def. 3.2, Prop. 3.4, Cor. 3.6	O domínio de δ não contém saída aprendida. A independência é propriedade da assinatura , e sobrevive a retreino e a mudança de prompt
Action Contract Object	ACO	Def. 3.1 (σ, ρ declarados), Prop. 3.5	A ação é declarada por inteiro antes da execução, e tirada de um catálogo fechado $\mathcal{C}$
Evidence-Bound Targeting	EBT	Def. 3.7, Req. 3.8, Def. 3.9	ext tem evidência estruturada por domínio. Texto livre não seleciona vítima porque não está no domínio
Principal-Perimeter Binding	PPB	E_1 , §5.1	$\text{ep}(a) \in \Pi(p)$ por igualdade estrita, com a credencial de escrita confinada a $\Pi(p)$. Sem perímetro padrão
Discriminator-Scoped Containment	DSC	Def. 5.4, Tm. 5.5, Cor. 5.7	Contenção por principal é expressável se e somente se existir predicado separador por principal. Não existindo, o principal é recusado em vez de registrado como defendido
Split-Authority Verification	SAV	E_4	A pós-condição é lida por credencial sem capacidade de escrita. Disjunção de capacidade, não de convenção
Sealed Session Chain	SSC	§4.2, Def. 4.1, Tm. 4.3	Digesto sobre conjunto ordenado fixo de campos, incluindo o do antecessor; digesto terminal selado sob autoridade de escrita separada
ProofPath Receipt	PPR	Def. 4.6, Req. 4.7, Cor. 4.4	O recibo carrega o bastante para um verificador que não importa código do emissor recomputar a cadeia — e localizar a alteração exatamente
Element-Level Reconciliation	ELR	Prop. 6.3, Cor. 6.4, §6.3	Sob agregação o mapa de origem não é recuperável, então a reconciliação chaveia pelo elemento contido, nunca pela regra do provedor
Review-Bound Testimony	RBT	§6.6	Evidência humana entra no registro e é lida por veredito determinístico. Nunca comanda
Sealed Review Loop	SRL	§6.5, Def. 6.9	A revisão pós-ação é função de decisão sobre (present, recurred, elapsed), registrada em sessão selada nova que cita a original sem reabri-la

Por que nomear sustenta peso aqui, e não é decoração

Três razões, e a terceira é a que importa.

Uma conjunção precisa de nome por si mesma. $E_1 \wedge E_2 \wedge E_3 \wedge E_4$ é um objeto com um modo de falha — qualquer conjunto falso, a ação é inadmissível — e uma prosa que soletra quatro predicados toda vez convida o leitor a tratá-los como lista de verificação da qual se pode negociar um. *AC* é a conjunção; não existe *AC* parcial.

Os pares fáceis de confundir ficam fáceis de separar depois de nomeados. SAV e PPB tratam os dois de credencial e são propriedades diferentes: PPB é sobre **onde** a credencial de escrita pode escrever, SAV sobre uma **segunda** credencial que não pode escrever em lugar nenhum. Um sistema pode satisfazer PPB e falhar SAV — confinar as escritas e então acreditar no próprio relato. A F1 da §9 é exatamente essa falha.

Mecanismo nomeado pode ser conferido quanto à existência; mecanismo sem nome é assumido como existente. Esta é a razão. Um artigo de arcabouço define doze mecanismos e uma implementação carrega onze, e a lacuna é invisível enquanto o décimo segundo é discutido em prosa que todo mundo lê como

descrição. Nomear torna o inventário finito e portanto auditável: dá para perguntar, a um sistema que se diz ADAI, quais dos doze ele implementa — e obter resposta que é **errada quando é errada**.

Os três que não são propriedade de um episódio

ELR, RBT e SRL não valem nem falham **dentro** de um episódio, e tratá-los como se valessem fabricaria veredito onde não há.

ELR é propriedade de uma passagem de reconciliação sobre o conjunto de efeitos em vigor, não de uma contenção. **RBT** só se aplica onde há testemunho, e a maioria dos episódios não tem nenhum. **SRL** fecha em sessão **diferente** da que está sendo revisada — por construção, já que reabrir a original violaria SSC.

Um relatório de conformidade sobre um episódio precisa, portanto, distinguir *não valeu* de *não se aplica aqui* e de *não decidível a partir deste registro*. Colapsar os dois últimos no primeiro fabrica falhas; colapsá-los em sucesso fabrica garantia. O segundo erro é o perigoso, e é o que um leitor quer por padrão.

8. Metodologia de avaliação

Distinguimos três famílias. Só a primeira é avaliação de aprendizado de máquina, e relatar apenas a primeira — que é a prática predominante da área — deixa as restrições que de fato governam a implantação sem medição.

8.1 Qualidade de política

Dado um sinal, o sistema propõe a ação correta? Medido contra um holdout cego de sessões reais com desfecho **verificado**, onde "correto" é definido pelo desfecho verificado e não pelo julgamento de um anotador.

Valem as métricas usuais, com um acréscimo. Defina o **erro ponderado por dano**

$$\mathcal{H} = \frac{1}{N} \sum_{i=1}^N w(\sigma_i) \cdot \mathbb{1}[a_i \neq a_i^*], \quad w(\sigma) = B(\sigma) (\Delta_{\text{det}} + \gamma (1 - \text{rev}(\sigma))),$$

com $\text{rev} \in [0, 1]$ um escore de reversibilidade, Δ_{det} o tempo esperado entre a ação errônea e a reversão dela, e $\gamma > 0$ um prêmio de irreversibilidade. γ tem unidade de tempo, de modo que w fica em *principal-segundos de negação indevida* — grandeza com significado operacional, diferente de um escore adimensional.

A forma aditiva é deliberada. O peso tentador $B(\sigma)(1 - \text{rev}(\sigma))$ atribui custo **zero** a um erro totalmente reversível, o que é falso: um bloqueio errado e reversível ainda nega serviço pelo intervalo entre o erro e a reversão, e esse intervalo é Δ_{det} — propriedade do laço de revisão, não da ação. Reversibilidade limita a *cauda* do dano; não elimina a *cabeça*. Uma métrica incapaz de enxergar a cabeça recompensa um sistema que reverte rápido e erra sempre.

Acurácia não ponderada, por sua vez, trata um endereço desafiado por engano e um provedor bloqueado por engano como um erro cada, que é precisamente a confusão identificada na §1.3.

O holdout precisa ser adversarial por construção: precisa conter sinais construídos para induzir ação errada, inclusive alguns cujos campos de texto livre nomeiem terceiros inocentes (testando o Req. 3.8), e o benefício da dúvida nunca deve ser dado.

8.2 Integridade do envelope

O sistema recusa o que o envelope precisa recusar? Isto não se mede numa amostra; mede-se por construção adversarial. Para cada predicado E_i , construa ações que o violam e confirme a recusa:

$$IE = \frac{\#\{\text{violações construídas recusadas}\}}{\#\{\text{violações construídas}\}}.$$

O alvo é $IE = 1$ exatamente, porque $\mathcal{E}$ é conjunção: uma única violação admitida significa que o predicado não é garantido, e uma taxa de 0,99 descreve um sistema sem garantia, não um sistema quase garantido.

8.3 Integridade da evidência

Três quantidades:

- **Taxa de recomputação:** fração de recibos exportados que um verificador de terceiro (Req. 4.7) valida.
- **Taxa de detecção de adulteração:** fração de cadeias sinteticamente alteradas que são rejeitadas.
- **Acurácia de localização de adulteração:** $\Pr[v(T') = j^*]$, esperada igual a 1 pelo Cor. 4.4; valor abaixo de 1 indica que a cadeia está funcionando como soma de verificação e não como cadeia.

8.4 O que não se pode medir, e precisa ser construído

Certas propriedades não admitem teste finito, porque o teste teria de enumerar um conjunto aberto:

- nenhuma credencial consegue escrever fora do seu perímetro;
- nenhum componente aprendido consegue ampliar o conjunto permitido;
- não existe caminho de texto livre para ação executada.

Cada uma é um enunciado universalmente quantificado sobre espaço ilimitado. São garantidas **por construção** — respectivamente por escopo de credencial no sentido de menor privilégio [32, 9, 23] (Def. 3.1, E_1), pela assinatura de δ (Prop. 3.4), e pelo domínio de ext (Req. 3.8) — e os testes correspondentes afirmam a *construção*, não a ausência de contraexemplos.

Esta posição — de que propriedades de segurança de sistemas que aprendem precisam ser garantidas arquiteturalmente e não estatisticamente — é defendida na literatura de segurança em aprendizado de máquina [2, 28]; damos dela uma instância concreta.

Esta é a afirmação metodológica central deste artigo. Para um agente que age sobre infraestrutura de produção, as propriedades que mais importam não admitem teste, e a arquitetura precisa portanto colocá-las onde testar é desnecessário. Um projeto em que essas propriedades são testáveis é um projeto em que elas são contingentes.

8.5 O que este artigo mede, e o que não mede

Não relatamos nada da §8.1. A implantação da §9 produziu poucos episódios de desfecho verificado demais para construir um holdout, e o holdout construído adversarialmente que aquela família exige — sinais forçados para induzir a ação errada — está especificado lá e não construído. Note-se que é artefato distinto do arnês de envelope abaixo: um ataca a **escolha**, o outro ataca a **permissão**, e construir o segundo nada estabelece sobre o primeiro. O leitor deve, portanto, tomar as §§3–7 como arquitetura com demonstrações e a §9 como um conjunto de contraexemplos, e não deve tomar nada neste artigo como evidência de que o agente resultante *escolhe bem*. Essa questão está aberta e não estamos em posição de fechá-la.

Relatamos a §8.2. Doze violações foram construídas sobre os quatro predicados — três ou mais por predicado — e todas as doze foram recusadas, dando $EIR = 12/12 = 1$. A construção inclui o ataque de proveniência de alvo do Req. 3.8 (um endereço nomeado apenas em campo de texto livre influenciado pelo

adversário), uma ação que declara escopo de efeito fora do perímetro defendido, uma ação em autonomia máxima com lista de rollback vazia, e uma tentativa de alargar o catálogo em que o nome de ação proposto não existe em $\mathcal{C}$.

Relatamos a §8.3, sobre um recibo exportado de uma sessão de produção de doze turnos, recomputado por um verificador que não importa parte alguma do sistema emissor e toma a função de hash, a lista ordenada de campos, as regras de serialização e a regra de encadeamento do próprio recibo. Taxa de recomputação 1; taxa de detecção de adulteração 1 sobre 36 alterações construídas — todo índice de turno sob cada uma das três estratégias adversariais do Tm. 4.3; e acurácia de localização 1 sobre as 23 alterações em que algum conjunto falhou.

As 13 restantes são relatadas à parte, e não dobradas em qualquer uma das taxas, porque dobrá-las em qualquer direção enuncia a propriedade errada. São os casos em que **nenhum conjunto falha por construção**: a cauda reescrita, e a alteração do **último** turno com o digesto recomputado, que não tem elo seguinte para carregar a descontinuidade. Nos dois, a detecção recai sobre o selo guardado à parte, e não há índice a localizar. Contá-las como falha de localização subestimaria uma cadeia que se comportou exatamente como o Tm. 4.3 prevê; excluí-las em silêncio esconderia a fronteira em que a cadeia deixa de localizar e passa a apenas detectar.

Estas duas famílias são mensuráveis precisamente por serem propriedades da construção e não de uma amostra, o que é também o que torna a omissão da §8.1 enunciável: os argumentos das §§3–7 não enfraquecem porque a qualidade de política está sem medição. Um arcabouço cujo argumento de segurança dependesse da acurácia do modelo não sobreviveria a essa admissão, e a §1.3 é a razão de ele não depender.

Uma ressalva pertence ao número do EIR e não a uma nota de rodapé. $\text{EIR} = 1$ autoriza afirmar que *as violações construídas são recusadas*. Não autoriza "o envelope é inviolável" — essa é afirmação universalmente quantificada sobre conjunto aberto, e a §8.4 é a razão de nenhum arnês estabelecê-la. Alargar o que o número autoriza exige acrescentar construções, não repetir a medição.

8.6 O que um resultado positivo autoriza

Menos do que parece. Qualidade de política autoriza afirmação sobre a distribuição de onde o holdout veio. Integridade de envelope autoriza afirmação sobre os predicados atacados. Integridade de evidência autoriza afirmação sobre o artefato, não sobre o mundo que ele descreve.

Nenhuma delas autoriza "é seguro rodar autonomamente". Essa afirmação repousa na conjunção de o envelope valer, o laço de revisão fechar, e o humano manter a capacidade de reverter — e a terceira não é medição, é compromisso operacional.

9. Classes de falha observadas

As seguintes foram observadas numa implantação de produção construída sobre este arcabouço, defendendo três serviços atrás de um domínio de aplicação compartilhado.

O estatuto probatório desta seção é o de contraexemplo, não o de amostra. Cada item estabelece que uma classe de falha é *alcançável* num sistema real com suíte de testes real — afirmação de existência, que uma instância resolve e que nenhuma quantidade adicional de instâncias fortaleceria. Não fazemos afirmação de frequência, e nenhum argumento das §§3–6 depende de uma. A seção serve para descartar a leitura de que essas classes são hipotéticas: a objeção "uma equipe cuidadosa não faria isso" é respondida por uma equipe cuidadosa tendo feito, com testes, e não tendo percebido.

Cada uma é *silenciosa* na ausência da garantia estrutural correspondente, e cada uma foi invisível aos testes existentes.

F1 — Isolamento conferido no caminho de leitura, assumido no caminho de ação. A filtragem por tenant era completa e testada em toda consulta; o caminho de ação resolvia o perímetro a partir da configuração do processo, e não do principal da sessão. A falha é inalcançável enquanto um único principal gera sinal e torna-se alcançável no instante em que um segundo é integrado. *Classe:* garantia auditada onde auditar é barato e assumida onde é caro. *Garantida por:* E_1 com igualdade estrita e sem padrão — o padrão "se não souber, use o perímetro da casa" é exatamente como o incidente de um principal vira indisponibilidade de outro.

F2 — Um recibo que ninguém conseguia recomputar. O recibo exportava uma projeção de exibição carregando 5 dos $k = 10$ campos hashados. A recomputação segundo as instruções do próprio recibo falhou em todos os turnos. Todos os testes internos passavam, porque todos construíam a forma do turno pelo código do próprio produtor. *Classe:* propriedade de um artefato, testada através do produtor. *Garantida por:* Def. 4.6(vi) e Req. 4.7.

F3 — Uma consulta por um valor que o escritor nunca escreve. A consulta de revisão selecionava turnos cuja literal de tipo diferia da persistida. A junção devolvia o conjunto vazio, em silêncio, por semanas: contenções executadas nunca eram revisadas, e a guarda contra readoção — usando a mesma literal — era permanentemente falsa, produzindo o modo de falso positivo do Cor. 6.4 a cada hora. Nenhum teste pegou, porque todos usavam dublê de armazenamento, e um dublê devolve o que o teste mandar. *Classe:* defeito na interface entre dois componentes, cada um individualmente correto e individualmente testado — risco conhecido dos dublês de teste, que codificam a crença do implementador sobre o outro lado em vez do outro lado [22]. *Garantida por:* derivar o valor de uma única declaração nas duas pontas, tornando a divergência erro de carga e não resultado vazio.

F4 — Uma garantia documentada que nunca foi imposta. Um registro de decisão afirmava que um portão de construção impedia dependências de modelo de terceiro no caminho de raciocínio. O portão conferia outra coisa. A ausência era de fato verdadeira — por disciplina, não por imposição. *Classe:* documentação que desliga a inspeção; quem lê "o portão cobre isso" para de conferir, e o custo se materializa não quando a afirmação fica falsa, mas quando alguém confia nela. *Garantida por:* a regra de processo de que toda garantia afirmada carrega uma checagem que a impõe, ou declara no próprio texto que é disciplina.

F5 — Divergência de nome de campo numa fronteira de ingestão. Dois caminhos de acesso aos mesmos eventos de um provedor usam capitalizações diferentes. O leitor usava uma; o caminho de entrega produzia a outra. Todo evento perdia o endereço do adversário, em silêncio — e o teste passava, porque o dublê misturava as duas grafias, forma que o provedor nunca emite. *Classe:* dublê que codifica a crença do implementador sobre um formato externo em vez do formato. *Garantida por:* fixtures capturadas literalmente do sistema externo.

F6 — Componentes agendados rodando código velho. O mecanismo de implantação atualizava serviços de longa duração e não tarefas agendadas, cada uma fixada no artefato que um humano registrou por último. A tarefa que fecha o laço de contenção e a tarefa que lê o feed de ameaças ficaram atrás da árvore testada — de modo que a correção de F5 foi mesclada, testada, implantada e nunca chegou ao componente que precisava dela. *Classe:* divergência entre o artefato testado e o artefato em execução, por um caminho que nenhum teste observa. *Garantida por:* um único mecanismo de implantação para todo componente que roda, relatando o que mudou.

F7 — Um predicado cuja implementação era mais estreita que o enunciado. E_2 exige $\text{effect}(a) \subseteq \sigma$ com σ limitado por $\Sigma_{\max}$, e a implementação o aplicava recusando alvos privados, de loopback, link-local, multicast ou reservados — os endereços que não são origem roteável. Ela admitia endereços pertencentes às

faixas publicadas do próprio ponto de aplicação. Duas contenções contra endereços de proxy compartilhado do provedor de aplicação ficaram em vigor numa zona de produção até serem encontradas por inspeção.

A parte instrutiva é o que **não** deu errado. A evidência era estruturada, não texto livre; a extração era sólida; o modelo propôs conter o endereço que a evidência nomeou, que era o endereço certo do ponto de vista dela — sob caminho proxiado, a origem observada é o proxy. Nada no caminho de raciocínio errou. A ação estava, ainda assim, fora de E_2 : um endereço de proxy compartilhado tem $B(\sigma) \gg 1$, já que o efeito recai sobre todo principal cujo tráfego chega por ele, e entre eles o próprio principal defendido. É, além disso, o caminho pelo qual a contenção seria observada, de modo que a verificação de E_4 e o efeito passam a depender um do outro.

Classe: um predicado enunciado corretamente e implementado como subconjunto próprio de si mesmo — a lacuna invisível porque o subconjunto implementado é o que um leitor confere. É a imagem espelhada de F4: lá, a documentação afirmava uma verificação que não existia; aqui, a verificação existia e era silenciosamente mais fraca que o predicado que se supunha cumprir. *Garantida por:* enumerar as faixas publicadas do ponto de aplicação como dado estático e datado no caminho de decisão — estático porque resolvê-las no momento da decisão faria δ depender de um serviço de rede e deixar de ser determinística, o que quebraria o contrato de recomputação da Def. 4.6 — e por acrescentar o caso ao arnês da §8.2, onde a ausência dele teria sido, de novo, invisível.

Estrutura comum. Nas sete, a garantia morava em convenção e não em construção, e o sintoma era ausência de sinal, não presença de erro. F7 afia o argumento: a garantia pode morar na construção e ainda assim falhar, se a construção implementar menos do que a garantia enuncia — que é por que a §8.2 mede os predicados atacando-os, e não lendo o código que os implementa.

Este é o argumento empírico a favor do arcabouço: não que a estrutura evite erros, mas que ela converte erros silenciosos em erros ruidosos.

10. Relação com trabalhos anteriores

Orquestração de segurança. Complementar e subsumida [4, 17]. Um catálogo revisado de playbooks é exatamente a forma certa para o conjunto permitido; a Prop. 3.5 preserva essa propriedade deliberadamente. O que o arcabouço acrescenta é um tratamento principiado de quando um agente pode escolher no catálogo sem ajuda, e do que ele deve depois.

Computação autônoma. O arcabouço é o laço MAPE-K [18, 16] com (i) um portão obrigatório entre Planejar e Executar cujo domínio exclui o conteúdo aprendido da base de Conhecimento, e (ii) um caminho de retorno obrigatório de Executar para Monitorar, com a direção mundo $\rightarrow$ registros da §6. Ambos estão no espaço que o MAPE-K deixa aberto.

Defesa cibernética aprendida. A §8.1 é a avaliação padrão desta literatura [37, 14, 25], adotada sem mudança. Nossa afirmação é que ela é uma família entre três e não é a restrição que governa a implantação.

Registro à prova de adulteração. A construção da §4.2 é padrão [13, 21, 35, 6, 20]. As contribuições são o contrato de recomputação da Def. 4.6, a observação de que a cláusula (vi) é rotineiramente afirmada e rotineiramente não cumprida (F2), e a localização exata do Cor. 4.4 como propriedade *mensurável* e não apenas afirmada.

Segurança por capacidades e segregação de funções. Segregação de funções é de Clark e Wilson [5]; capacidades, de Dennis e Van Horn [9], com o tratamento moderno em Miller [23]. E_4 é segregação de funções realizada como disjunção de capacidades e não como atribuição de papel [33]. A distinção é que atribuição de papel é propriedade do programa — que uma refatoração quebra em silêncio, deixando uma verificação que continua retornando sucesso e não prova nada — enquanto disjunção de capacidades é propriedade da credencial, que a refatoração não altera.

Multi-tenancy. A literatura de isolamento concentra-se em isolamento de plano de dados e de armazenamento [19]. A §5 trata do isolamento de *efeitos sobre infraestrutura compartilhada*, onde o recurso compartilhado é o próprio ponto de aplicação, e o Teo. 5.5 dá a condição de expressabilidade. Não temos conhecimento de tratamento equivalente.

11. As dificuldades, resolvidas

Um rascunho anterior deste artigo terminava com uma lista de problemas em aberto. Aquela lista é esta seção, e nada nela está aberto. Cada dificuldade é enunciada como condição com construção, e onde a condição falha o arcabouço prescreve o que o sistema precisa recusar afirmar — o que é, em si, um resultado, e o a que este artigo mais recorre.

Expomos aqui em vez de deixá-las como agenda, porque um arcabouço que nomeia uma dificuldade e não volta a ela nomeou uma lacuna em si mesmo. O que sobrevive é medição e não método, e a §11.5 diz quais medições e por que elas não são a mesma coisa que problema em aberto.

11.1 Reconciliação contra um mundo de granularidade alheia

A §6 resolveu o caso em que efeitos se agregam em regras. O problema geral é: quando registros podem ser reconciliados contra um mundo cujas unidades de observação não são as unidades de efeito do sistema?

Definição 11.1 (Atomização). Seja O o estado observável de um ponto de aplicação e $\mathcal{F}$ o conjunto de efeitos que o sistema consegue criar. Uma *atomização* é um mapa computável $\alpha : O \rightarrow 2^{\mathcal{A}}$ para um conjunto de átomos, junto de um mapa $\iota : \mathcal{A} \rightarrow \mathcal{F} \cup \{\perp\}$, tal que (i) cada átomo é derivável da observação apenas, sem recorrer aos registros do sistema; e (ii) ι é injetiva sobre os átomos com $\iota \neq \perp$ — cada átomo corresponde a no máximo um efeito que o sistema poderia ter criado.

Teorema 11.2 (Condição de reconciliação). A reconciliação mundo $\rightarrow$ registros é correta num ponto de aplicação se e somente se existe uma atomização. Sob uma, o algoritmo é: computar $\alpha(O)$ e, para cada átomo com $\iota(\text{átomo}) \neq \perp$, testar os registros em busca do efeito correspondente; os átomos que falham o teste são órfãos.

Demonstração. ($\Leftarrow$) A cláusula (i) torna a travessia computável pelo lado do mundo, que é o que falta à revisão guiada por registros. A cláusula (ii) torna o teste bem posto: um átomo mapeia para no máximo um efeito, então "este efeito está registrado?" tem resposta determinada. A completude decorre de que todo efeito criado pelo sistema está em O e produz, portanto, um átomo. ($\Rightarrow$) Suponha a reconciliação correta. Então, para cada observação, existe um procedimento que decide, por efeito criado, se ele está registrado. A partição de O induzida pelas decisões daquele procedimento é uma atomização: é computada de O apenas, e as classes dela correspondem a no máximo um efeito cada, pois do contrário dois efeitos compartilhariam uma decisão e um poderia estar registrado e o outro não. $\square$

Corolário 11.3 (Onde a reconciliação é impossível, e o que fazer). Se o ponto de aplicação expõe apenas estado agregado do qual efeitos individuais não são recuperáveis — um contador, uma taxa, um booleano "proteção ligada" — então não existe atomização, e a reconciliação é impossível, não apenas não implementada.

A resposta correta é a que a §5.2 já prescreve para principais indefensáveis: recusar a capacidade em vez de afirmá-la. Um sistema que integre tal ponto de aplicação precisa registrar que os efeitos dele ali são **irreconciliáveis**, e não pode relatar uma passada de reconciliação sobre eles como limpa. Relatar "O órfãos" quando órfãos são indetectáveis é a frase mais danosa que um sistema desses pode emitir, porque é indistinguível da verdadeira.

Os três arranjos da §5.3 são agora instâncias. Por contenção, α é a identidade sobre regras e ι lê a descrição da regra. Por principal, α decompõe a regra no conjunto de endereços dela e ι mapeia cada endereço para a contenção daquele endereço para aquele principal — que é precisamente por que a reconciliação teve de migrar para o elemento (Prop. 6.3). Sob a regra única compartilhada, α precisa recuperar endereço e principal de uma expressão só; ela consegue, de modo que a reconciliação segue possível — o que vale notar, porque significa que o arranjo compartilhado é rejeitado pelas razões da §5.3 e não por esta.

11.2 Correlação entre perímetros sem divulgação entre perímetros

Um adversário sondando q principais é uma campanha só. A evidência que revelaria isso está, por E_1 , particionada por principal. O dilema aparente é que correlação exige comparar evidência através da partição.

Não exige. Correlação exige comparar *um valor derivado* por observação, e o valor pode ser escolhido de modo a não carregar nenhuma outra informação.

Construção 11.4 (Índice cego de correlação). Fixe uma chave κ detida pela instalação e por principal algum. A cada contenção do alvo x para o principal p , a sessão emite, num índice append-only compartilhado fora da partição de todo principal, a tupla

$$\langle \text{PRF}_{\kappa}(x), \text{carimbo de tempo, tipo de sinal} \rangle$$

e **nenhum identificador de principal**. A correlação é então o cômputo de multiplicidades sobre o índice: um valor que aparece k vezes com $k > 1$ indica uma origem contida em k perímetros.

Proposição 11.5 (O que o índice esconde e o que não esconde). Para um leitor que não detenha κ nem as pré-imagens do índice, o índice revela o *número* de perímetros que uma origem alcançou e os horários, e não revela nem a origem nem os principais. Para um leitor que detenha κ , ele revela a origem: o espaço de endereços é pequeno o bastante para ser enumerado, de modo que PRF_{κ} não esconde de quem tem a chave.

Demonstração. Sem κ , $\text{PRF}_{\kappa}(x)$ é indistinguível de aleatório pela hipótese de PRF , e a tupla não carrega campo de principal, logo nenhum principal é identificável. Com κ , computar $\text{PRF}_{\kappa}(x')$ para todo x' do espaço de endereços recupera x por consulta em tabela; para IPv4 são 2^{32} avaliações, o que não é barreira. $\square$

Isso delimita a construção com precisão, e o limite é a parte útil: o índice protege a evidência de um principal **de outro principal**, que é a propriedade de que E_1 trata, e não a protege do operador, que já detém as credenciais de todos os perímetros e portanto já vê toda contenção. Implantar o índice com κ em poder do operador não é, portanto, enfraquecimento de E_1 — não concede a ele nada que lhe faltasse — e dá aos principais um sinal agregado que eles não obteriam de outro modo.

Duas propriedades mantêm a construção dentro do arcabouço. O índice é **append-only e fora de toda partição**, de modo que escrever nele não é leitura entre tenants. E ele não carrega **identificador de principal**, de modo que mesmo quem tem a chave aprende "esta origem atingiu k perímetros" e não quais — distinção que importa se o índice for algum dia divulgado, e que não custa nada preservar.

A limitação residual é honesta e estreita: se q for pequeno, a contagem em si identifica, já que "3 de 3 perímetros" os nomeia. Abaixo de um limiar em q a construção não deve ser implantada, e o limiar é decisão de implantação, não criptográfica.

11.3 Escassez de corpus: a conta é melhor do que a que demos

Escrevemos que o sinal de aprendizado são sessões com desfecho verificado, que a barra é alta, e que poucos exemplos se acumulam por implantação. As duas primeiras orações estão certas e a terceira subestima o corpus por confundir *contenções* com *exemplos rotulados*.

Cada contenção produz uma sequência de rótulos, não um. O laço de revisão (§6.5) revisita toda contenção periodicamente e emite um veredito a cada vez: reafirmada, cedo demais, expirada, ausente, ou falso positivo. Uma contenção que viva T horas sob revisão horária contribui com da ordem de T observações rotuladas da mesma decisão sob evidência mutável — e as *transições* entre vereditos são os exemplos mais informativos do corpus, por serem exatamente os momentos em que o mundo contradisse um julgamento anterior.

Recusas são dado de desfecho verificado e são muito mais abundantes que contenções. Todo sinal que a curadoria rejeita, toda extração que não oferece alvo acionável, todo tipo de sinal ausente do catálogo, e toda negação da função de decisão é um episódio registrado com desfecho determinado e motivo escrito.

Isso foi afirmado num rascunho anterior e agora está medido. Numa janela de trinta dias na implantação da §9, com todos os identificadores omitidos:

Grandeza	Contagem
Episódios no tenant do operador	114
— encerrados sem ação	81
— selados	33
Turnos de recusa, janela de 30 dias	93
— atribuíveis a um único critério (severidade abaixo do limiar de materialidade)	79 (85%)
Sinais cuja evidência nunca ofereceu alvo acionável	10
Contenções com desfecho revisado	4

Duas coisas nesta tabela importam mais que a razão. A primeira é que as recusas superam as contenções por cerca de uma ordem de grandeza, o que resolve a questão aritmética e torna a composição do conjunto de treino o oposto do que se imaginaria. A segunda são os 85%: um *único* critério responde pela maior parte do que o sistema deixa de olhar, o que significa que aquele critério, sozinho, define o campo de visão do sistema. É achado sobre a implantação e não sobre o arcabouço, e é o tipo de achado que a camada de medição existe para trazer à superfície — ele foi produzido pela própria camada de achados do sistema, e não por um analista consultando o banco.

Treinar um modelo para narrar apenas o que foi contido descartaria, portanto, cerca de nove décimos do corpus, e os nove décimos mais informativos: a fronteira do conjunto permitido é desenhada pelas recusas, não pelas ações.

Adoções também são rotuladas. Um efeito adotado (§6.2) entra na revisão e acumula vereditos identicamente, apesar de não ter recibo de execução.

O que permanece genuinamente aberto é mais estreito que "escassez de corpus", e o enunciamos na forma estreita: se a *distribuição* de um corpus assim composto — dominado por recusas, com contenções como classe minoritária e transições de veredito ainda mais raras — sustenta o treino de um narrador que não degrade na

classe minoritária. É questão de desbalanceamento de classe, com literatura substancial e nenhum resultado específico a este arcabouço, e é questão empírica que este artigo não resolve.

11.4 Ações compostas entre domínios de aplicação

$\mathcal{E}$ é enunciado para um perímetro. Uma ação que atravessasse dois domínios com predicados expressáveis diferentes e credenciais diferentes exige regra de composição, e a dificuldade é E_3 : o rollback de uma composta não é, em geral, a composição dos rollbacks.

Seja uma **ação composta** uma sequência ordenada $a = \langle a_1, \dots, a_n \rangle$ com a_i agindo no domínio Z_i , cada a_i carregando o próprio rollback declarado ρ_i e o próprio escopo de efeito σ_i .

Teorema 11.6 (Reversibilidade de composta). Suponha que cada a_i satisfaça E_3 individualmente e que os escopos de efeito sejam dois a dois disjuntos: $\sigma_i \cap \sigma_j = \emptyset$ para $i \neq j$. Então a composta satisfaz E_3 com rollback $\rho = \langle \rho_n, \dots, \rho_1 \rangle$ e, além disso, ρ restaura a identidade em $\bigcup_i \sigma_i$ independentemente de qual prefixo de a tenha executado.

Demonstração. A disjunção torna os $\text{exec}(a_i)$ dois a dois comutantes em $\bigcup_i \sigma_i$, já que cada um age como identidade fora do próprio escopo. Cada $\text{exec}(\rho_i) \circ \text{exec}(a_i) = \text{id} \upharpoonright_{\sigma_i}$ por hipótese. Para um prefixo $a_1 \dots a_m$, aplicar $\rho_m, \dots, \rho_1$ dá id em $\sigma_1 \cup \dots \cup \sigma_m$ e, por disjunção, deixa $\sigma_{m+1} \dots \sigma_n$ intocados — onde nada foi feito. A ordem inversa não é exigida pela comutatividade e sim pela disciplina de registro de E_3 , para que uma compensação nunca preceda na transcrição a ação que ela compensa. $\square$

Corolário 11.7 (Condição de recusa). Se $\sigma_i \cap \sigma_j \neq \emptyset$ para algum $i \neq j$, a composta não satisfaz E_3 em geral e precisa ser recusada no ponto de decisão, em vez de tentada com rollback de melhor esforço.

Justificativa. Com escopos sobrepostos, ρ_j pode restaurar um estado da sobreposição que a_i havia deliberadamente mudado, de modo que a composição não é a identidade; qual dos dois prevalece depende da ordem de execução, e um rollback cujo resultado depende do intercalamento não é rollback. $\square$

Três consequências decorrem para a implementação, e são o conteúdo prático do teorema.

A disjunção é checável antes da execução, a partir dos escopos declarados, que é por que E_2 exige o escopo declarado. O ponto de decisão ganha, portanto, uma regra: para uma composta, conferir a disjunção dois a dois dos escopos declarados e recusar em caso contrário.

A cadeia inteira de compensação é registrada antes da primeira execução, como E_3 já exige por ação; o registro da composta é a concatenação, e precisa estar durável antes de a_1 rodar. Uma queda em qualquer ponto deixa uma transcrição da qual as compensações restantes são computáveis.

A verificação compõe sem regra nova. Cada a_i é verificado em Z_i pela credencial somente-leitura daquele domínio, e a composta é verificada se e somente se todo componente o for. Nenhum verificador entre domínios é necessário, o que importa porque um verificador entre domínios precisaria de alcance com capacidade de escrita em vários domínios — o oposto de E_4 .

11.5 Desbalanceamento de classe: limitado por construção, não por esperança

A §11.3 reduziu "escassez de corpus" a uma questão mais estreita: se um corpus dominado por recusas treina um narrador que não degrada nas classes minoritárias — contenções e transições de veredito. Deixada ali, é incógnita empírica. Não precisa ficar ali, porque a arquitetura já proíbe o desfecho ruim de chegar à produção.

A construção exige duas coisas da implantação, e as enunciamos como hipóteses em vez de citá-las, porque nenhuma foi estabelecida antes neste artigo: **(G1)** a publicação de um narrador treinado é condicionada a uma medição tomada *por classe de desfecho* contra uma linha de base determinística, sendo a publicação

recusada se alguma classe cair abaixo dela; e **(G2)** na ausência de modelo publicado, a narração cai para aquela regra determinística.

Proposição 11.8 (A degradação não chega à produção). Sob G1 e G2, para qualquer distribuição de treino, o narrador implantado ou é ao menos tão bom quanto a regra determinística em toda classe, ou não há narrador implantado e o sistema narra por regra.

Demonstração. Por G1 o portão é pré-condição da publicação e a medição dele é condicional à classe, de modo que um modelo que degrade em qualquer classe reprova e não é publicado. Por G2 o ramo restante narra pela regra determinística, cujo comportamento independe do corpus e portanto da distribuição de classes dele. Os dois ramos satisfazem a afirmação. $\square$

G1 e G2 são propriedades que uma implantação tem ou não tem, e o artigo companheiro relata um sistema que tem as duas.

A proposição não afirma que o modelo será bom. Afirma que o **risco é unilateral**: desbalanceamento de classe pode custar um narrador à implantação, e não pode custar-lhe uma narração errada. Essa assimetria é o que torna a questão não bloqueante, e ela existe porque o narrador não decide (critério C) — um componente aprendido excluído da decisão de admissibilidade pode falhar sem que a falha se propague para um efeito.

Duas construções melhoram as chances, em vez de apenas limitar a perda.

Estratificação ponderada por dano. Exemplos de treino são ponderados pelo custo operacional $w(\sigma)$ da §8.1, e não pela frequência. O peso já existe, já está definido em unidade com significado operacional, e codifica exatamente o julgamento de que uma contenção errada importa mais que uma recusa errada. Usar frequência otimizaria para a classe com que o sistema menos se importa.

Transições como classe própria. A §11.3 observou que as transições de veredito são os exemplos mais informativos. São também os mais raros, de modo que sob ponderação por frequência elas desaparecem. Tratá-las como classe de desfecho distinta, com limiar próprio no portão, impede que um modelo passe por ir bem no regime estacionário e mal exatamente nos momentos que o registro existe para capturar.

O que sobra é medição, não problema: os números condicionais à classe, quando existir corpus de tamanho suficiente. A §8.5 já diz que este artigo não relata medição de qualidade de política, e a razão é a mesma.

11.6 Pontos de aplicação que não atomizam: tornar a falha ruidosa

O Cor. 11.3 mostrou que um ponto de aplicação que expõe apenas estado agregado não admite atomização, e prescreveu recusar a capacidade de reconciliação. A recusa é correta e incompleta: ela deixa o sistema incapaz de defender tal ponto de todo, se defendê-lo honestamente exige reconciliação.

Duas construções fecham isso, e elas trocam uma pela outra.

Construção 11.9 (Atomizar recusando a agregação). Onde o ponto de aplicação expõe estado por objeto mas o sistema havia agregado vários efeitos num objeto, a atomização se recupera alocando um objeto de aplicação por efeito — o arranjo por contenção da §5.3. A reconciliação passa então a ser identificada por regra e correta.

Isso é exato e custa capacidade: $\mathcal{R} = N$ contra um orçamento de $L - k$ (Prop. 5.9). É a escolha certa onde o orçamento permitir, e o argumento da §5.3 a favor da agregação vale apenas onde ele não permite.

Construção 11.10 (Registro de intenção antecipado). Onde o ponto de aplicação não expõe estado por efeito nenhum — um contador, uma taxa, um booleano —, alocação alguma recupera a atomização, porque a deficiência está em O e não no uso que o sistema faz dele. Introduza, em vez disso, um segundo repositório I ,

sob autoridade de escrita distinta tanto do ponto de aplicação quanto da transcrição. Antes de $\text{exec}(a)$, o sistema anexa a I um registro de *intenção* nomeando o ponto de aplicação, o efeito pretendido e a sessão. Depois de uma compensação verificada, a intenção é marcada como quitada.

A reconciliação passa a correr sobre I e não sobre O : uma intenção não quitada sem registro de sessão correspondente é um órfão.

Proposição 11.11 (O que o registro de intenção recupera e o que não recupera). Com I , todo efeito que o sistema tentou num ponto de aplicação não atomizável é detectável a partir de I apenas. Uma intenção não quitada **não** estabelece que o efeito passou a valer, já que O não pode ser consultado por efeito; ela estabelece que o sistema tentou e que não existe registro do desfecho.

Demonstração. A intenção é escrita antes da execução e está, portanto, presente para toda tentativa, inclusive aquelas cuja escrita de sessão falhou — que é a janela identificada pela §6.1. Reciprocamente, I registra intenção e não observação, logo não consegue distinguir um efeito que passou a valer de um cuja execução falhou no ponto de aplicação, distinção que exige exatamente a consulta por efeito que O não suporta. $\square$

O ganho é precisamente o que este arcabouço persegue em toda parte: converte um órfão **indetectável** em uma ambiguidade **detectável**. Antes, o sistema não podia dizer nada e não sabia que não estava dizendo nada. Depois, ele pode dizer "houve tentativa de efeito aqui, o desfecho dela não está registrado, e este ponto de aplicação não me diz qual foi" — frase sobre a qual um operador consegue agir, e que o sistema é obrigado a emitir em vez de relatar passada limpa.

Corolário 11.12 (A regra de relato). Um sistema que integre ponto de aplicação não atomizável relata três contagens, e não uma: reconciliadas, órfãs e **inverificáveis**. Colapsar a terceira em qualquer das duas primeiras é o erro de que o Cor. 11.3 adverte, e o registro de intenção é o que torna a terceira contagem computável em vez de apenas concebível.

Juntas, as Construções 11.9 e 11.10 não deixam ponto de aplicação algum em que o sistema tenha de mentir ou de se abster. Onde o orçamento permitir, atomizar exatamente. Onde não permitir e O for por objeto, agregar e reconciliar por elemento (§6.3). Onde O for agregado, usar o registro de intenção e relatar a terceira contagem.

12. Conclusão: a defesa autônoma é possível, e é assim

A pergunta que este artigo se propôs a responder era se uma máquina pode ter permissão de defender infraestrutura por conta própria — decidindo, no momento do incidente, sem humano no laço, e agindo sobre sistemas que carregam tráfego alheio. Argumentamos que a resposta é sim, que é sim sob condições declaradas e não em geral, e que as condições são estruturais, e não questão de quão bom acontece de ser o componente que raciocina.

Os resultados das §§3–11 se juntam numa única afirmação construtiva, e a damos nessa forma porque um arcabouço de proibições lido como lista de proibições deturpa aquilo para que ele serve.

Teorema 12.1 (Viabilidade da ação defensiva autônoma). Seja S um sistema que satisfaça o seguinte, cada item estabelecido por uma seção anterior:

1. **A evidência é ancorada.** Alvos são extraídos apenas por uma função total de parsing sobre um conjunto fixo de chaves da evidência estruturada do sinal, excluindo linguagem natural de autoria do adversário (Req. 3.8).

2. **O conjunto permitido é fixado antes de o componente que raciocina rodar.** Ações são instanciadas de um catálogo revisado, e o domínio da função de admissibilidade exclui toda saída aprendida (Prop. 3.4).
3. **Toda ação está no envelope.** Perímetro vinculado ao principal sem padrão, escopo de efeito declarado e limitado por constante de política, rollback registrado antes da execução, e pós-condição verificada por credencial de capacidade disjunta (Def. 3.1).
4. **A contenção por principal é expressável.** O ponto de aplicação admite um predicado separador para cada principal defendido (Teo. 5.5); principais para os quais ele não admite têm o registro recusado (Cor. 5.7).
5. **Todo efeito é alcançável a partir dos registros.** Existe uma atomização, ou um registro de intenção antecipado supre uma (Teo. 11.2, Constr. 11.10).
6. **Toda ação é registrada de forma verificável.** Transcrição encadeada por hash, selada sob autoridade de escrita separada, exportada com contrato de recomputação (Def. 4.6).

Então S pode agir sem aprovação humana concomitante com as seguintes garantias, todas válidas independentemente da qualidade de qualquer componente aprendido:

- **(a) Dano limitado.** O $\max B(a)$ sobre toda ação que S consegue executar, sob todo comportamento de modelo, é constante de implantação computável sem rodar o sistema (Cor. 3.4a).
- **(b) Nenhum efeito entre principais.** Toda contenção casa dentro do tráfego de exatamente um principal (Teo. 5.5), e uma sessão cujo principal não tem perímetro registrado não age (§5.2).
- **(c) Estado recuperável.** Todo efeito tem rollback registrado antes dele, e ações compostas entre domínios são reversíveis exatamente quando os escopos declarados são disjuntos — condição conferida antes da execução (Teo. 11.6).
- **(d) Nenhum efeito silencioso.** Todo efeito criado é alcançável a partir dos registros ou relatado como inverificável; nenhum é invisível (Teo. 11.2, Cor. 11.12).
- **(e) Prestação de contas conferível por terceiro.** Qualquer parte pode verificar o registro sem o código do sistema e localizar qualquer alteração exatamente (Teo. 4.3, Req. 4.7).

Demonstração. Cada garantia é o resultado citado sob a hipótese citada; a conjunção vale porque as hipóteses são independentes — (1) restringe a extração, (2) a construção, (3) a admissibilidade, (4) a expressabilidade, (5) a observabilidade, (6) o registro — e nenhuma demonstração recorre à negação de outra. $\square$

O que isso autoriza operacionalmente

Lido contra as fases do tratamento de incidentes [4], o teorema cobre três delas e para exatamente onde para o alcance da implantação, o que vale enunciar em vez de borrar.

A contenção é direta. As garantias (a)–(c) são exatamente o necessário para permitir que uma máquina altere um ponto de aplicação sem supervisão: o dano está limitado antes do fato, não consegue alcançar outro principal, e pode ser retirado. É a fase que o arcabouço trata de forma mais completa.

A erradicação é limitada pelo alcance do ponto de aplicação, e não a superestimamos. Um sistema que age numa borda de rede remove o *acesso* de um adversário, não a presença dele: encerra o alcance, e não consegue limpar um host comprometido. O que a garantia (d) acrescenta é a parte que a erradicação de fato exige e que a maior parte da automação não tem — que a resposta seja *completa*, com todo efeito contabilizado. O acesso de um adversário não é erradicado por um bloqueio que algum processo posterior reverteu em silêncio e ninguém notou. O laço de revisão (§6.5) é o que faz da remoção um fato verificado continuamente em vez de uma escrita única, e a travessia de reconciliação (§6.2, §11.6) é o que torna "não há efeitos esquecidos" uma afirmação conferível em vez de uma esperança.

A **prevenção é o produto de segunda ordem do laço**. Todo sinal recusado, todo playbook ausente e todo alvo não acionável é registrado com o motivo (§3.1, e §12 do artigo do sistema), e esses registros são a entrada da camada de medição: o catálogo cresce onde estavam as lacunas, sob revisão humana. Entre perímetros, o índice cego (Constr. 11.4) torna a campanha de um adversário visível como uma campanha, sem que a evidência de principal algum saia da partição dele. Nenhum dos dois é predição; ambos são a maneira ordinária pela qual uma defesa melhora, tornada computável.

Por que a falibilidade do componente que raciocina não é fatal

O movimento que faz tudo isso funcionar é uma separação, e vale enunciá-lo com todas as letras porque é a afirmação central do artigo sobre *como* a autonomia se torna admissível.

Um agente defensivo precisa de julgamento: quais sinais importam, qual alvo é o adversário, qual resposta cabe. Julgamento desse tipo é o domínio de componentes falíveis, treinados sobre dados, e cuja entrada é em parte escrita pelo adversário. Construir um argumento de segurança sobre a acurácia de um componente desses é construí-lo sobre a cooperação do adversário.

Então o sistema não constrói. O **conjunto** de ações admissíveis é fixado pelo catálogo e pelo envelope antes de qualquer raciocínio rodar, e é conferível sem rodá-lo. A **escolha dentro do conjunto** é onde mora o julgamento, e onde um componente aprendido pode contribuir. Um modelo errado escolhe uma ação permitida que um modelo melhor não teria escolhido; ele não escolhe uma inadmissível, porque as inadmissíveis não estão no conjunto do qual ele escolhe.

É isso que separa um sistema autônomo de um não supervisionado. O primeiro tem uma fronteira que alguém desenhou deliberadamente e pode inspecionar; o segundo tem a fronteira que o componente mais falível dele acabar produzindo. As falhas da §9 são de duas formas, e ambas silenciosas pela mesma razão: seis em que a garantia morava em convenção e não em construção, e uma em que ela morava na construção mas a construção implementava menos do que a garantia enunciava. A segunda forma é a mais difícil, e é a razão de a §8.2 atacar os predicados em vez de ler o código que os implementa — uma fronteira que se pode inspecionar só vale a pena ser desenhada se inspecionar for outra coisa que reler a própria intenção.

A afirmação, dita com exatidão

Uma máquina pode ter permissão de prevenir, conter e prestar contas de ataques a infraestrutura de produção, por conta própria, quando: age apenas sobre evidência que ela extraiu estruturalmente e verificou de forma independente; a fronteira do que ela pode fazer foi desenhada antes de ela rodar e não se move; todo efeito que ela cria está vinculado a um principal, é reversível, e é alcançável a partir de um registro probatório; e esse registro pode ser conferido por quem não confia nela.

Essas condições não são fáceis, e a contribuição deste artigo é que elas estão *enunciadas* — como teoremas com demonstração, com as fronteiras em que cada uma falha e a instrução de recusar ali a capacidade em vez de afirmá-la. Um sistema que as cumpra falha alto onde falha — fail-stop em vez de fail-silent [34, 10] — e é isso, e não infalibilidade, que uma máquina que age sobre a infraestrutura alheia deve a quem depende dela.

Limitações

As classes de falha da §9 vêm de uma única implantação de produção, ao longo de alguns dias. São instâncias, não estatísticas; o valor probatório está na *classe* de cada falha e na invisibilidade demonstrada de cada uma aos testes existentes, e não em qualquer afirmação de frequência. Os Teoremas 4.3 e 5.5 são incondicionais

dadas as hipóteses declaradas; as Props. 3.4 e 3.5 são arquiteturais e valem apenas para sistemas construídos como especificado. Não se afirma que o arcabouço seja suficiente para segurança, apenas que cada um de seus componentes é necessário para a garantia que nomeia.

As medições da §8.5 carregam os próprios limites. $EIR = 1$ é sobre doze construções, e um arnês mede a própria cobertura: um predicado só fica estabelecido na medida em que os ataques a ele foram imaginados. Os números de integridade de evidência são sobre um único recibo — um censo completo dos índices de turno e das estratégias adversariais, e portanto exatos para aquela cadeia, mas uma cadeia — de modo que estabelecem que o contrato de recomputação é *satisfazível e satisfeito ali*, e não uma taxa sobre uma população. Nenhuma das duas famílias fala da §8.1, e o leitor que lê $EIR = 1$ como evidência de que o agente age *bem* leu a quantidade errada: ela é evidência de que ele não pode agir **fora**.

Referências

As entradas são mantidas na língua original das publicações.

- [1] Alahmadi, B. A., Axon, L., Martinovic, I. (2022). *99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms*. Proceedings of the 31st USENIX Security Symposium.
- [2] Biggio, B., Roli, F. (2018). *Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning*. Pattern Recognition 84, 317–331.
- [3] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM 59(5), 50–57.
- [4] Cichonski, P., Millar, T., Grance, T., Scarfone, K. (2012). *Computer Security Incident Handling Guide*. NIST Special Publication 800-61 Revision 2.
- [5] Clark, D. D., Wilson, D. R. (1987). *A Comparison of Commercial and Military Computer Security Policies*. IEEE Symposium on Security and Privacy, 184–194.
- [6] Crosby, S. A., Wallach, D. S. (2009). *Efficient Data Structures for Tamper-Evident Logging*. Proceedings of the 18th USENIX Security Symposium.
- [7] Cvach, M. (2012). *Monitor Alarm Fatigue: An Integrative Review*. Biomedical Instrumentation & Technology 46(4), 268–277.
- [8] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352.
- [9] Dennis, J. B., Van Horn, E. C. (1966). *Programming Semantics for Multiprogrammed Computations*. Communications of the ACM 9(3), 143–155.
- [10] Gray, J. (1986). *Why Do Computers Stop and What Can Be Done About It?* Symposium on Reliability in Distributed Software and Database Systems, 3–12.
- [11] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M. (2023). *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 79–90.
- [12] Guo, C., Pleiss, G., Sun, Y., Weinberger, K. Q. (2017). *On Calibration of Modern Neural Networks*. Proceedings of the 34th International Conference on Machine Learning, 1321–1330.

- [13] Haber, S., Stornetta, W. S. (1991). *How to Time-Stamp a Digital Document*. Journal of Cryptology 3(2), 99–111.
- [14] Hammar, K., Stadler, R. (2020). *Finding Effective Security Strategies through Reinforcement Learning and Self-Play*. 16th International Conference on Network and Service Management (CNSM).
- [15] Hanappi, O., Hummer, W., Dustdar, S. (2016). *Asserting Reliable Convergence for Configuration Management Scripts*. OOPSLA, 328–343.
- [16] IBM (2006). *An Architectural Blueprint for Autonomic Computing*, 4th edition. IBM White Paper.
- [17] Islam, C., Babar, M. A., Nepal, S. (2019). *A Multi-Vocal Review of Security Orchestration*. ACM Computing Surveys 52(2), Article 37.
- [18] Kephart, J. O., Chess, D. M. (2003). *The Vision of Autonomic Computing*. IEEE Computer 36(1), 41–50.
- [19] Krebs, R., Momm, C., Kounev, S. (2012). *Architectural Concerns in Multi-Tenant SaaS Applications*. 2nd International Conference on Cloud Computing and Services Science (CLOSER), 426–431.
- [20] Laurie, B., Langley, A., Kasper, E. (2013). *Certificate Transparency*. RFC 6962, IETF.
- [21] Merkle, R. C. (1988). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87, LNCS 293, 369–378.
- [22] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [23] Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Tese de doutorado, Johns Hopkins University.
- [24] Moreau, L., Groth, P. (2013). *Provenance: An Introduction to PROV*. Morgan & Claypool.
- [25] Nguyen, T. T., Reddi, V. J. (2023). *Deep Reinforcement Learning for Cyber Security*. IEEE Transactions on Neural Networks and Learning Systems 34(8), 3779–3795.
- [26] OASIS (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard.
- [27] Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., Snoek, J. (2019). *Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift*. Advances in Neural Information Processing Systems 32.
- [28] Papernot, N., McDaniel, P., Sinha, A., Wellman, M. P. (2018). *SoK: Security and Privacy in Machine Learning*. IEEE European Symposium on Security and Privacy, 399–414.
- [29] Parasuraman, R., Sheridan, T. B., Wickens, C. D. (2000). *A Model for Types and Levels of Human Interaction with Automation*. IEEE Transactions on Systems, Man, and Cybernetics — Part A 30(3), 286–297.
- [30] Perez, F., Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*. arXiv:2211.09527.
- [31] SAE International (2021). *J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*.
- [32] Saltzer, J. H., Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1278–1308.
- [33] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., Youman, C. E. (1996). *Role-Based Access Control Models*. IEEE Computer 29(2), 38–47.
- [34] Schlichting, R. D., Schneider, F. B. (1983). *Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems*. ACM Transactions on Computer Systems 1(3), 222–238.

- [35] Schneier, B., Kelsey, J. (1999). *Secure Audit Logs to Support Computer Forensics*. ACM Transactions on Information and System Security 2(2), 159–176.
- [36] Sheridan, T. B., Verplank, W. L. (1978). *Human and Computer Control of Undersea Teleoperators*. Technical Report, MIT Man-Machine Systems Laboratory.
- [37] Standen, M., Lucas, M., Bowman, D., Richer, T. J., Kim, J., Marriott, D. (2021). *CybORG: A Gym for the Development of Autonomous Cyber Agents*. arXiv:2108.09118.
- [38] Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., Thomas, C. B. (2018). *MITRE ATT&CK: Design and Philosophy*. MITRE Technical Report MP180360.
-

Autoria

Hiago Kin Levi

Brooklyn, NY — 16 de setembro de 2026, 22h00

SHA-256 do fonte Markdown, até a linha marcadora abaixo:

```
d24484ff5345fc8536627292b4b1957428d12e943614d5ac5a8c9eea8dba7038
```